Lab 4: A TX UART Controller

Yuval Zecharia

15/10/2026

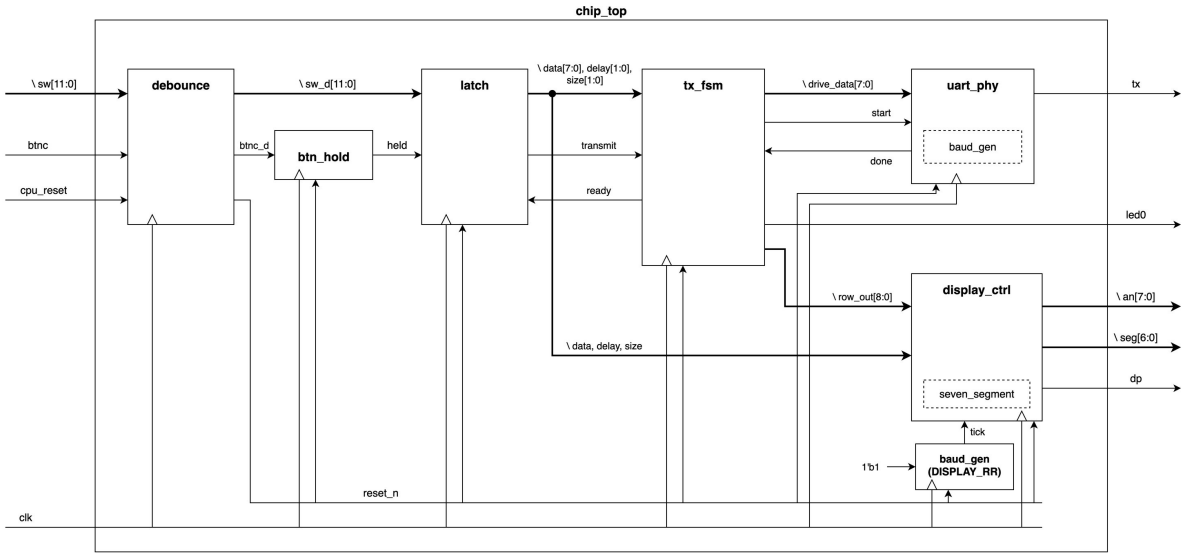
1. מטרת המעבדה

תכנון, מימוש ובדיקה של בקר UART TX ללוח ה-Nexys A7-100T. הבקר שולח למחשב ריבוע של N×N עותקים של הבית שנבחר במתגים, ומציג את התצורה ואת מספר השורות שנשלחו על תצוגת שבעת המקטעים. דרישות המעבדה:

1. כניסות: SW[7:0] – הבית לשליחה, SW[9:8] – השהיה בין בתים (0, 50, 100 או 200ms), SW[14:13] – גודל הריבוע (1, 32, 128 או 256), הכפתור המרכזי ו-CPU RESET (פעיל בנמוך). יציאות: קו ה-TX ו-LED[0], שמתהפך בכל בית נתון (Byte0 דולק, Byte1 כבוי וכן הלאה).
2. UART PHY בקצב 57600bps, LSB תחילה.
3. TX FSM עם מצב המתנה ייעודי בין בתים ויציאות רשומות (עדיפות ל-3 בלוקי always ול-Moore). אחרי כל בית נשלח 8'h20, ואחרי הבית האחרון בשורה נשלחים 8'h0D ו-8'h0A.
4. תצוגה ללא הבהוב: T0 – הבית, T1 – ההשהיה (0.0, 0.5, 1.0, 2.0 עם נקודה עשרונית), T2 – מספר הבתים לשליחה (N), T3 – מספר השורות שנשלחו; ב-T2 וב-T3 יש לשים לב להגבלת הטווח.
5. דגימת המתגים לאחר החזקת הכפתור המרכזי במשך יותר משנייה.
6. מונה בתים משודרים (ללא התווים המיוחדים) שערכו מוצג ב-T3.
7. בקר תצוגה בקצב רענון של 500Hz, המונע משעון מחולק מ-100MHz.
8. טסטבנצ'ים המרחיבים את אלה של המעבדה הקודמת, עם timescale 1ns/1ps וזמן לחיצה של 100ns בסימולציה.
9. מימוש פיזי: קובץ XDC, סינתזה, Place & Route, יצירת bitstream ובדיקה על הלוח.

2. תהליך התכנון

2.1. ארכיטקטורה



איור 1: מבנה chip_top: תת-המודולים והאותות ביניהם.

כל הכניסות האסינכרוניות עוברות דרך debounce. החזקה של שנייה ב-BTNC יוצרת פולס held, ו-latch דוגם את המתגים רק כש-tx_fsm פנוי (ready). tx_fsm מפיק את רצף הבתים ומעביר כל בית ל-uart_phy בלחיצת יד start/done. display_ctrl מציג את התצורה ואת row_out. כל המודולים עובדים על שעון יחיד של 100MHz; האירועים האיטיים מיוצגים על ידי פולסי tick המשמשים כ-enable.

טבלה 1: תת-המודולים של העיצוב ותפקידם.

Module	File	Function
chip_top	src/chip_top.v	Top level: instances and wiring, parameters for hardware / simulation
debounce	src/debounce.v	2-FF synchronizer + 20 ms debounce for CPU_RESETN (power-on reset_n) and for {SW, BTNC}
btn_hold	src/btn_hold.v	FSM: one-clock held pulse after BTNC was held for 1 s
latch	src/latch.v	Latches data, delay, size on held & ready; one-clock transmit pulse
tx_fsm	src/tx_fsm.v	Byte-level FSM: data, 0x20, 0x0D 0x0A, inter-byte WAIT, col/row counters, LED0
uart_phy	src/uart_phy.v	Bit-level 8N1 transmitter: 10-bit shift register and bit counter
baud_gen (x2)	src/baud_gen.v	One-clock tick every 100 MHz / rate: baud tick in uart_phy, digit tick for the display
display_ctrl	src/display_ctrl.v	8-digit scan: anode ring, T0–T3 decode, dp on AN3, FF saturation
seven_segment	src/seven_segment.v	Hex digit to 7 active-low cathodes

2.2. ממשקי המודולים

טבלה 2: ממשק chip_top.

Port name	Direction	Type	Description
clk	Input	Wire	100 MHz system clock (E3)
cpu_reset	Input	Wire	CPU_RESETN push-button, active low
btnc	Input	Wire	Center push-button: 1 s hold latches SW and starts a run
sw	Input	Wire [11:0]	{SW14, SW13, SW9..SW0} = {size, delay, data}
tx	Output	Wire	UART serial line to the PC (UART_RXD_OUT, D4)
led0	Output	Wire	LED[0], toggles on every data byte
an	Output	Wire [7:0]	Digit anodes, active low, one digit on at a time
seg	Output	Wire [6:0]	Cathodes {CA..CG} of the active digit, active low
dp	Output	Wire	Decimal point, on (0) while AN3 is active

טבלה 3: הפרמטרים של chip_top וערכיהם בחומרה ובסימולציה.

Parameter	Hardware value	Simulation (tb_chip_top)	Meaning
BAUD_RATE	57600	10,000,000	uart_phy bit rate – equation (1)
DISPLAY_RR	4000	10,000,000	Digit tick of the display – equation (3)
MAX_COUNT_DEBOUNCE	2,000,000	4	Stable time of the debounce (20 ms) – equation (4)
MAX_COUNT_BTN_HOLD	100,000,000	10 (100 ns)	BTNC hold time (1 s) – equation (5)
DELAY50	5,000,000	20	Clocks of the 50 ms delay – equation (6)

טבלה 4: ממשק debounce.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
cpu_reset	Input	Wire	Raw CPU_RESETN (asynchronous)
btnc	Input	Wire	Raw center button (asynchronous)
sw	Input	Wire [11:0]	Raw switches (asynchronous)
sw_d	Output	Wire [11:0]	Synchronized, debounced switches
btnc_d	Output	Wire	Synchronized, debounced button
cpu_reset_d	Output	Wire	reset_n for the whole design; INIT 0 gives a power-on reset

טבלה 5: ממשק btn_hold.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
btn	Input	Wire	btnc_d from debounce
held	Output	Wire	One-clock pulse when the button was held for MAX_COUNT clocks

טבלה 6: ממשק latch.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
ready	Input	Wire	tx_fsm is in IDLE
sw_d	Input	Wire [11:0]	Debounced switches
held	Input	Wire	1 s pulse from btn_hold
data / delay / size	Output	Reg [7:0] / [1:0] / [1:0]	Latched configuration, kept for the whole run
transmit	Output	Reg	One-clock start of a run, one clock after the latch

טבלה 7: ממשק tx_fsm.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
data / delay / size	Input	Wire [7:0] / [1:0] / [1:0]	Configuration from latch
transmit	Input	Wire	Starts a run (IDLE only)
done	Input	Wire	uart_phy finished the current byte
drive_data	Output	Reg [7:0]	Byte for uart_phy: data, 0x20, 0x0D or 0x0A
start	Output	Reg	One-clock request to uart_phy
led0	Output	Reg	Toggles on every data byte
row_out	Output	Wire [8:0]	Rows transmitted, 0..256
ready	Output	Wire	state == IDLE

טבלה 8: ממשק uart_phy.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
data	Input	Wire [7:0]	Byte to transmit
start	Input	Wire	Loads {1, data, 0} in IDLE
tx	Output	Wire	Serial line, LSB first; 1 when idle
done	Output	Wire	One-clock pulse at the end of the stop bit

טבלה 9: ממשק baud_gen.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
en	Input	Wire	Counter runs while 1 and restarts at 0 when 0
baud_tick	Output	Wire	One-clock tick every 100 MHz / BAUD_RATE clocks

טבלה 10: ממשק display_ctrl.

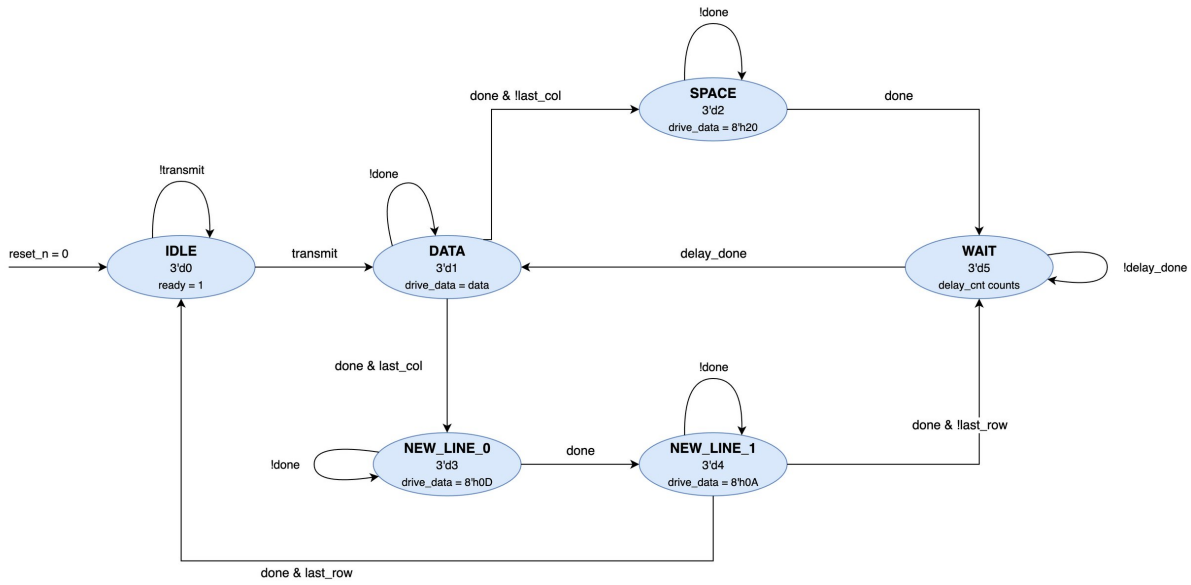
Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronous reset (active low)
tick	Input	Wire	Digit tick – advances the anode ring
data / delay / size	Input	Wire [7:0] / [1:0] / [1:0]	Values for T0, T1, T2
row	Input	Wire [8:0]	row_out for T3 (FF when row[8] = 1)
an	Output	Reg [7:0]	Anode ring, starts at 8'hFE
seg	Output	Wire [6:0]	Cathodes from seven_segment
dp	Output	Wire	an[3]: decimal point of T1

טבלה 11: ממשק seven_segment.

Port name	Direction	Type	Description
dig_in	Input	Wire [3:0]	Hex digit 0–F
ss_dig	Output	Reg [6:0]	Active-low cathodes {CA..CG}

2.3. מכונות המצבים

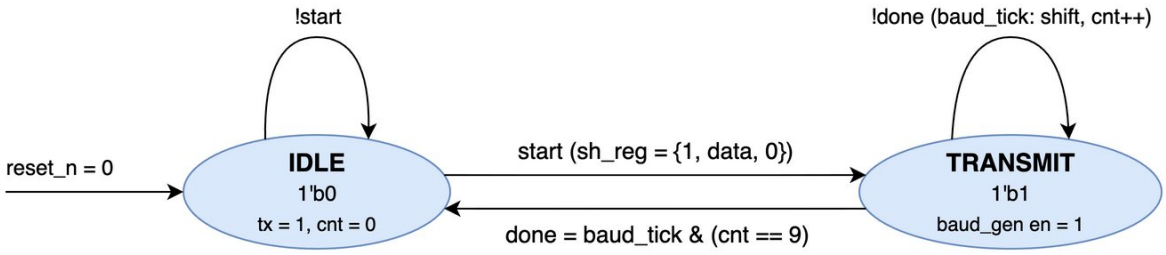
בעיצוב שלוש מכונות מצבים. בכולן רגיסטר המצב ולוגיקת המצב הבא כתובים בבלוקי always נפרדים, והמונים ורגיסטרי היציאה בבלוקים משלהם. בטבלאות המעבר, כל שורה מתארת חץ אחד ואת מה שנטען ברגיסטרים באותו מחזור; מצב נשמר כל עוד אף תנאי יציאה ממנו אינו מתקיים.



איור 2: דיאגרמת המצבים של tx_fsm.

טבלה 12: טבלת המעברים של tx_fsm.

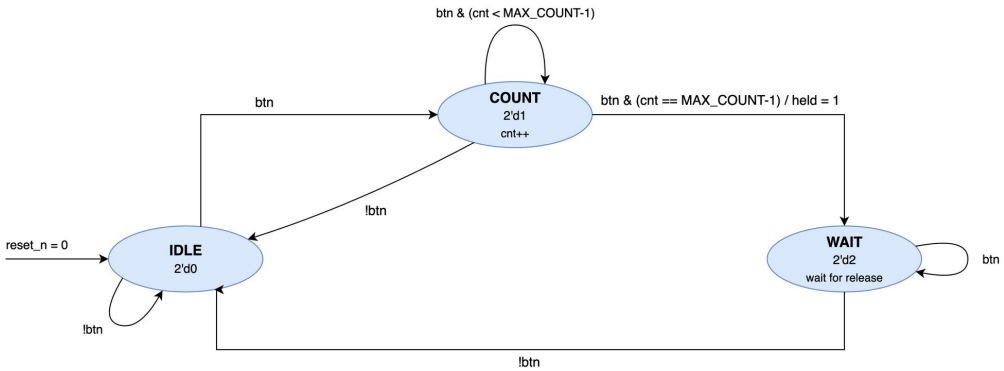
Current state	Condition	Next state	Registered on the transition
IDLE (0)	transmit	DATA	start = 1, drive_data = data, led0 toggles; col = row = 0
DATA (1)	done & !last_col	SPACE	start = 1, drive_data = 8'h20
DATA (1)	done & last_col	NEW_LINE_0	start = 1, drive_data = 8'h0D
SPACE (2)	done	WAIT	col++, delay_cnt = 0
NEW_LINE_0 (3)	done	NEW_LINE_1	start = 1, drive_data = 8'h0A
NEW_LINE_1 (4)	done & !last_row	WAIT	row++, col = 0, delay_cnt = 0
NEW_LINE_1 (4)	done & last_row	IDLE	row++ (row_out = N)
WAIT (5)	delay_done	DATA	start = 1, drive_data = data, led0 toggles



איור 3: דיאגרמת המצבים של uart_phy.

טבלה 13: טבלת המעברים של uart_phy.

Current state	Condition	Next state	Registered on the transition
IDLE (0)	start	TRANSMIT	sh_reg = {1, data, 0}: tx = 0 (start bit)
TRANSMIT (1)	baud_tick & cnt < 9	TRANSMIT	sh_reg = {1, sh_reg[9:1]}, cnt++
TRANSMIT (1)	done = baud_tick & cnt == 9	IDLE	sh_reg = all 1, cnt = 0



איור 4: דיאגרמת המצבים של btn_hold.

טבלה 14: טבלת המעברים של btn_hold.

Current state	Condition	Next state	Registered on the transition
IDLE (0)	btn	COUNT	cnt = 0
COUNT (1)	btn & cnt < MAX_COUNT-1	COUNT	cnt++
COUNT (1)	!btn	IDLE	cnt = 0
COUNT (1)	btn & cnt == MAX_COUNT-1	WAIT	cnt = 0; held = 1 in this clock
WAIT (2)	!btn	IDLE	–

ב-tx_fsm היציאות start, drive_data ו-led0 נטענות על תנאי החץ (האותות load_data, load_space ו-load_new_line_x), ולכן הן רשומות ומגיעות מחזור אחד אחרי המעבר. כך אין צורך בארבעה מצבי LOAD נפרדים כמו במימוש Moore טהור. held של btn_hold הוא יציאה קומבינטורית של מחזור אחד, והוא נרשם ב-latch כ-transmit. ב-debounce אין מכונת מצבים: המונה עצמו מחזיק את המצב, וכל שינוי בכניסה מאפס אותו.

2.4. שיקולי תכנון

- קצב השידור: baud_gen סופר עד COUNT ומפיק baud_tick רק במצב TRANSMIT (en = state). החלוקה השלמה נותנת שגיאת קצב זניחה (משוואה 1), ופריים של 10 ביטים נמשך 173.6μs (משוואה 2).

$$COUNT = \frac{f_{clk}}{BAUD_RATE} = \frac{100\ MHz}{57600} \approx 1736,\ f_{baud} = \frac{100\ MHz}{1736} \approx 57604\ bps\ (+0.006\%) \quad (1)$$

$$T_{frame} = 10 \cdot COUNT \cdot T_{clk} = 10 \cdot 1736 \cdot 10\ ns = 173.6\ \mu s \quad (2)$$

- תצוגה: מופע שני של baud_gen עם DISPLAY_RR=4000 מפיק tick של 4kHz, והטבעת של 8 ספרות מקבלת רענון של 500Hz (משוואה 3). ה-tick משמש כ-enable ולא כשעון מחולק, ולכן אין תחום שעון נוסף. dp מחובר ל-an[3] ודולק רק בספרה השמאלית של T1, כך שמתקבל 0.0, 0.5, 1.0 או 2.0.

$$f_{tick} = \frac{f_{clk}}{25000} = 4\ kHz,\ f_{refresh} = \frac{f_{tick}}{8} = 500\ Hz \quad (3)$$

- debounce וסנכרון: כל כניסה אסינכרונית עוברת שני דלגלגים, גלאי שינוי ומונה יציבות של 20ms (משוואה 4). ל-CPU_RESETN מונה נפרד עם ערכי INIT, ולכן n_reset יוצא 0 אחרי טעינת ה-bitstream (power-on reset). המתגים והכפתור הם וקטור אחד של 13 ביטים עם מונה משותף.

$$MAX_COUNT_DEBOUNCE = f_{clk} \cdot 20\ ms = 2 \cdot 10^6 \ \Rightarrow\ \lceil log_2(2 \cdot 10^6) \rceil = 21\ bit \quad (4)$$

- החזקה של שנייה: btn_hold סופר MAX_COUNT מחזורים כל עוד btnc_d=1 (משוואה 5), ומפיק held אחד בכל לחיצה. המצב WAIT מחכה לשחרור, ולכן החזקה ארוכה אינה מפעילה שידור נוסף.

$$MAX_COUNT_BTN_HOLD = f_{clk} \cdot 1\ s = 10^8 \ \Rightarrow\ \lceil log_2 10^8 \rceil = 27\ bit \quad (5)$$

- השהיה בין בתים: delay מפוענח ל-DELAY100 ,DELAY50 ,0 או DELAY200, ורוחב delay_cnt נקבע לפי הערך הגדול (משוואה 6). ב-delay=0 התנאי delay_done מתקיים מיד, ומצב WAIT נמשך מחזור אחד.

$$DELAY200 = 4 \cdot f_{clk} \cdot 50\ ms = 2 \cdot 10^7 \ \Rightarrow\ \lceil log_2 (DELAY200 + 1) \rceil = 25\ bit \quad (6)$$

- תווים מיוחדים ומונים: col סופר בתי נתון בשורה ו-row סופר שורות. אחרי בית שאינו אחרון בשורה נשלח 0x20, ואחרי האחרון נשלחים 0D ו-0A, גם בשורה האחרונה. מכאן מספר הבתים בריצה (משוואה 7). אחרי השורה האחרונה אין WAIT.

$$N_{bytes} = N \cdot (N + (N-1) + 2) = N \cdot (2N + 1) \ \Rightarrow\ N = 32{:}\ 2080,\ N = 256{:}\ 131328 \quad (7)$$

- מונה השידור: סעיף 4 ב-PDF מגדיר את T3 כמספר השורות שנשלחו, וסעיף 6 מגדיר מונה של בתי נתון. מספר הבתים מגיע עד 65536 ואינו נכנס בשתי ספרות hex, ולכן T3 מציג את row_out, שעולה פעם אחת בכל N בתי נתון. row ברוחב 9 ביטים כדי להכיל 256. T2 ו-T3 מציגים FF כשהערך הוא 256 (row[8]=1), שהוא הגבלת הטווח של שתי ספרות hex.
- latch: התצורה נדגמת רק כש-(load_cfg = held & ready) ready=1, כך שלחיצה באמצע ריצה אינה משנה אותה. transmit הוא load_cfg מושהה במחזור אחד, ולכן tx_fsm קורא כבר את הערכים החדשים.
- אילוצי תזמון: הכניסות מסונכרנות בתוך העיצוב, והיציאות (LED, תצוגה ו-UART ב-57600bps) איטיות ביחס לשעון. לכן הוגדרו להן set_false_path בקובץ ה-XDC במקום input/output delay, וכך דוח ה-Methodology נקי מאזהרות TIMING-18.
- סימולציה: הפרמטרים מוקטנים רק בטסטבנץ' (טבלה 3), וערכי החומרה נשארים בקוד. לחיצה מגיעה ל-held אחרי 17 מחזורים: 2 מחזורי סנכרון, מחזור של גלאי השינוי, MAX_COUNT_DEBOUNCE−1 מחזורי יציבות ומחזור של רגיסטר היציאה, ואחריהם מחזור כניסה ל-COUNT ו-MAX_COUNT_BTN_HOLD−1 מחזורי ספירה (משוואה 8). held מתקבל רק מלחיצה של 11 מחזורים לפחות (משוואה 9). ה-debounce מזיז את שני הקצוות של הלחיצה באותה כמות ולכן אינו משנה את הרוחב הנדרש.

$$t_{held} = (2 + 1 + (4-1) + 1) + (1 + (10-1)) = 7 + 10 = 17\ clk \quad (8)$$

$$L_{min} = MAX_COUNT_BTN_HOLD + 1 = 11\ clk \quad (9)$$

2.5. קוד המקור

קוד המקור של תשעת המודולים (תיקיית src) ושורות האילוצים הפעילות מקובץ ה-XDC מובאים בנספח א'.

2.6. סביבת הבדיקה

כל טסטבנץ' בודק את עצמו: הוא מחשב את הפלט הצפוי, מדפיס שורת PASS/FAIL לכל בדיקה ומסיים בשורת TEST PASS/FAIL. כל הקבצים משתמשים ב-timescale 1ns/1ps ובשעון של 10ns.
tb_chip_top בנוי כמו סביבת UVM מצומצמת: driver (drive, btn_press), monitor על tx שדוגם באמצע כל ביט, predictor (build_exp) ו-scoreboard שמזהה בית שגוי, בית מיותר ובית חסר.

טבלה 15: הטסטבנצ'ים ומה שכל אחד מהם בודק.

Testbench	DUT	What is checked	Checks
tb_uart_phy	uart_phy (10 clk/bit)	frame {1, data, 0} sampled mid-bit by a UART receiver model, done after exactly 100 clocks; 0x00, 0xAA, 0x55, 0xFF, 10 random bytes, back-to-back bytes	32
tb_tx_fsm	tx_fsm + uart_phy (DELAY50=20)	every byte of the run (data, 0x20, 0x0D 0x0A), row_out, run time incl. WAIT; 1x1 and 32x32 with all four delays	8338
tb_chip_top	chip_top (sim parameters)	1x1 0x41; short press – no TX; 32x32 delay 1; press during a run ignored; reset during a run (tx, led0, ready, row_out, no more frames)	4179

קוד הטסטבנצ'ים מובא בנספח ב'; הפלט של כל טסטבנץ' מוצג בפרק 4 לצד צורות הגל.

3. בדיקת Lint

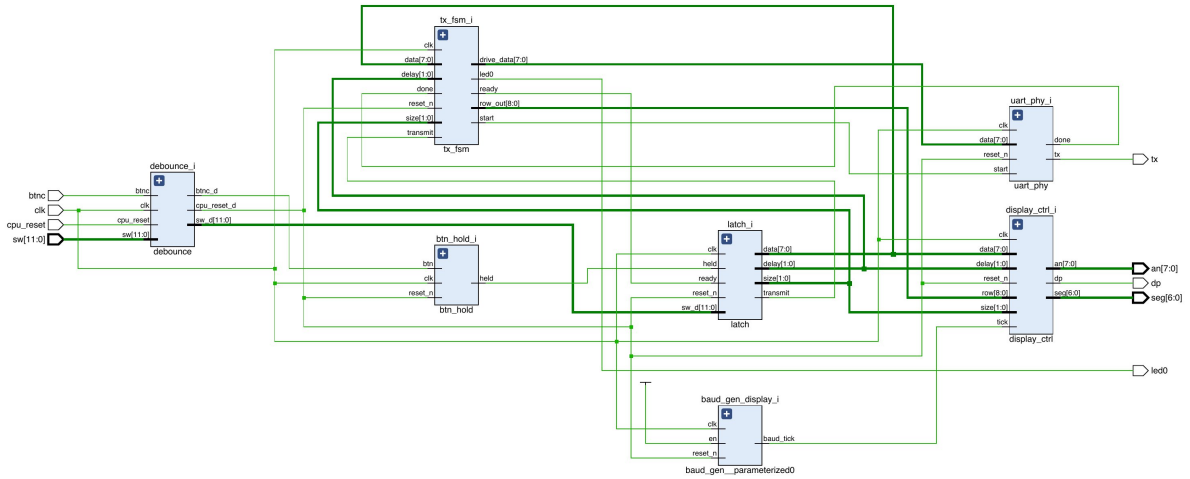
ה-Linter של Vivado (RTL ANALYSIS → Run Linter) הורץ על chip_top יחד עם כל תת-המודולים ולא מצא הפרות. ב-Elaborated Design דוח ה-DRC נקי, ובדוח ה-Methodology מופיעה הערת Advisory אחת (RFFI-1): an_reg[3] מזין גם את an[3] וגם את dp, ו-Vivado מציע לשכפל אותו. החיבור מכוון, ובמימוש opt_design אכן שכפל את רגיסטרי ה-an.

טבלה 16: תוצאות בדיקות ה-Lint וה-Elaborated Design.

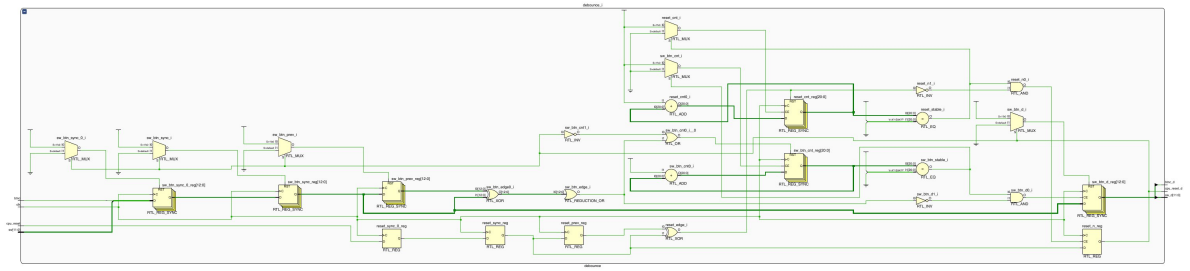
Check	Scope	Result	Report file
Vivado RTL Linter (Run Linter)	chip_top + all sub-modules	0 violations	vivado_lint_report_chip_top.txt
Elaborated design – Report DRC	chip_top (397 instances)	0 checks	elaborated_design_drc.rpt
Elaborated design – Report Methodology	chip_top	1 advisory: RFFI-1 (an_reg[3] drives an[3] and dp)	elaborated_design_methodology.rpt

3.1. סכמות RTL (Elaborated Design)

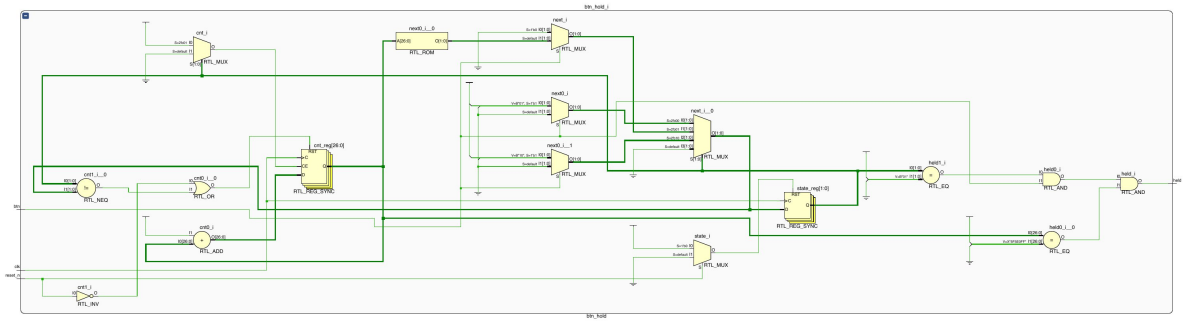
סכמות ה-elaboration של כל המודולים, כפי שהן מוצגות ב-Schematic → RTL ANALYSIS:



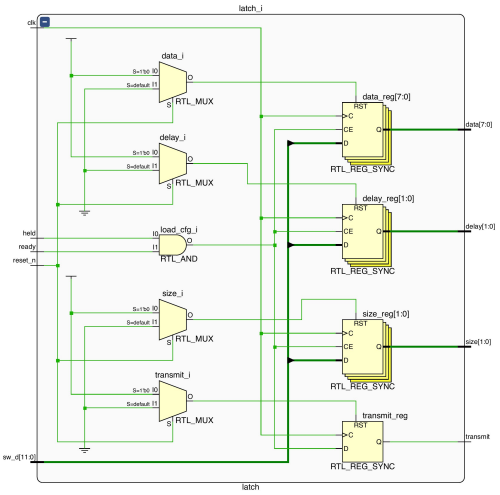
איור 5: סכמת ה-RTL של chip_top: debounce, btn_hold, latch, tx_fsm, uart_phy, baud_gen של התצוגה ו-display_ctrl.



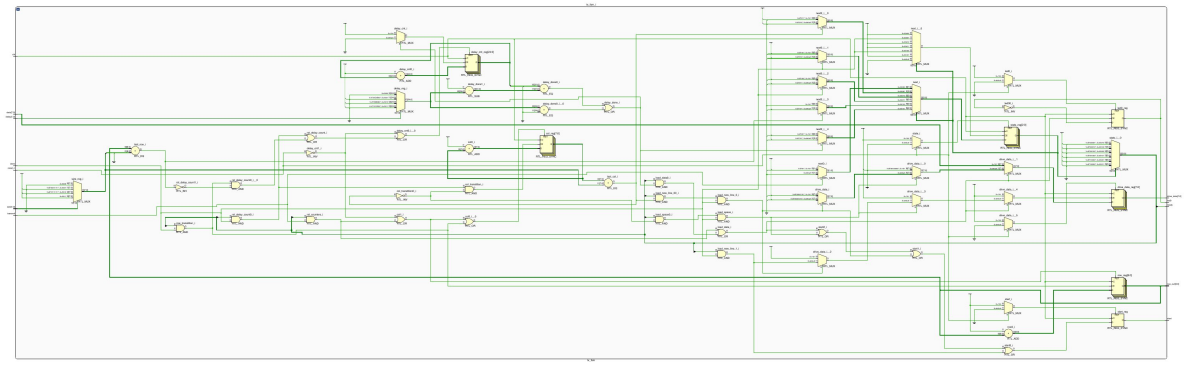
איור 6: סכמת ה-RTL של debounce: סנכרון, גלאי שינוי, מונה יציבות ורגיסטר יציאה לכל אחד משני הענפים (reset ו-{SW, BTNC}).



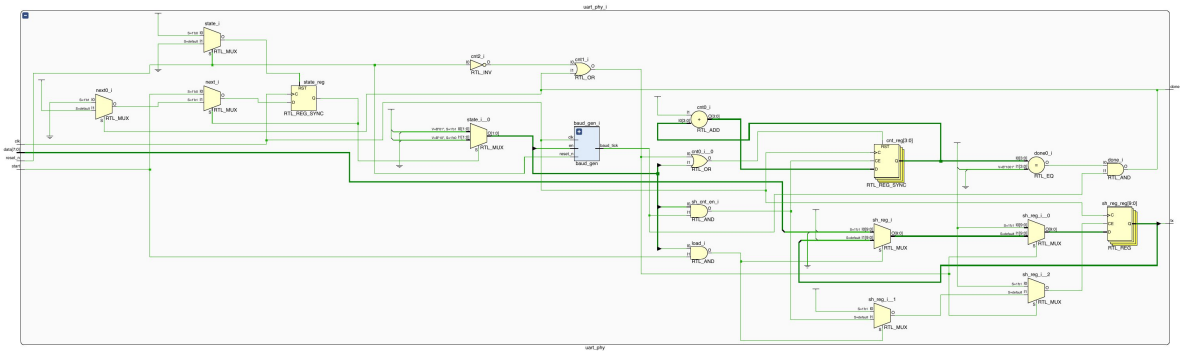
איור 7: סכמת ה-RTL של btn_hold: רגיסטר המצב, לוגיקת המצב הבא, המונה וה-AND של held.



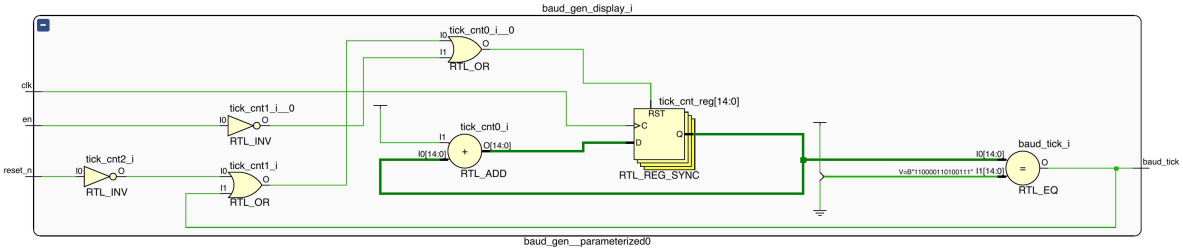
איור 8: סכמת ה-RTL של latch: רגיסטר התצורה עם enable של load_cfg ודלגלג transmit.



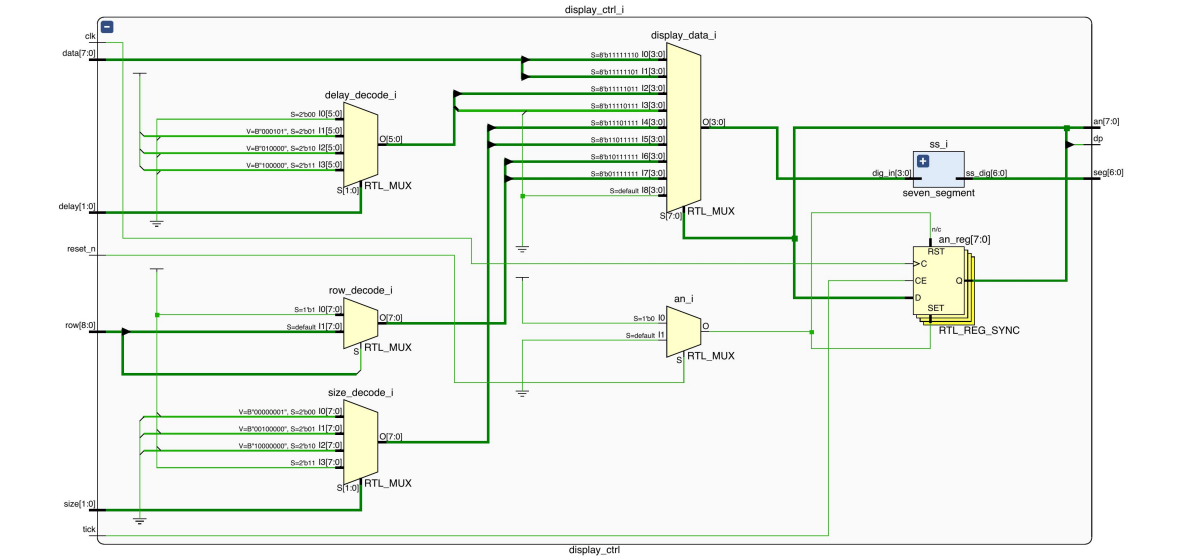
איור 9: סכמת ה-RTL של tx_fsm: מפענחי delay ו-size, המונים col, row ו-delay_cnt, מכונת המצבים ורגיסטרי היציאה.



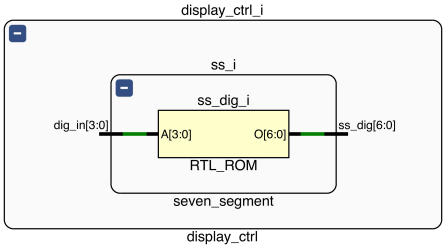
איור 10: סכמת ה-RTL של uart_phy: baud_gen, מונה הביטים, רגיסטר ההזזה ורגיסטר המצב.



איור 11: סכמת ה-RTL של baud_gen: מונה, משווה ואיפוס.



איור 12: סכמת ה-RTL של display_ctrl: טבעת האנודות, המפענחים של T1–T3, הבורר ו-seven_segment.



איור 13: סכמת ה-RTL של seven_segment: משפט ה-case כ-ROM.

4. תוצאות סימולציה

כל הטסטבנצ'ים הורצו ב-xsim Vivado 2025.2. האיורים הם צילומי מסך של חלון ה-Waveform של Vivado, עם סמנים (markers) ממוספרים על האירועים שמוסברים אחרי כל איור. לפני האיורים מובא פלט הטסטבנץ'.

טבלה 17: סיכום תוצאות הסימולציה.

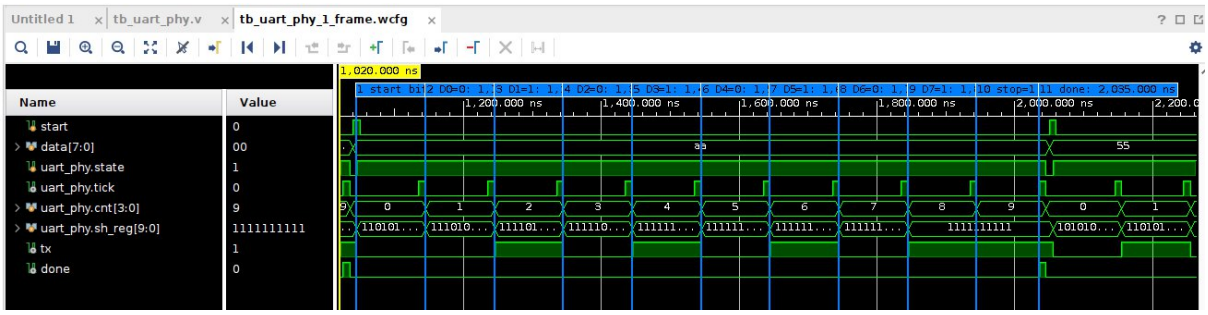
Testbench	DUT	Checks	Result	Sim time
tb_uart_phy	uart_phy (BAUD 10M)	32/32	TEST PASS	16,235 ns
tb_tx_fsm	tx_fsm + uart_phy (DELAY50=20)	8338/8338	TEST PASS	9,851,835 ns
tb_chip_top	chip_top (sim parameters)	4179/4179	TEST PASS	4,437,105 ns
Total	3 testbenches	12549/12549	0 failures	

4.1. tb_uart_phy – בדיקת uart_phy

פלט הטסטבנץ' (tb_uart_phy_xsim.log):

```
Time: 985 | Frame correctly recived | actual=1000000000 | expected=1000000000 | PASS
Time: 1035 | Clock cycles during transmition | actual=100 | expected=100 | PASS
Time: 1995 | Frame correctly recived | actual=1101010100 | expected=1101010100 | PASS
Time: 2045 | Clock cycles during transmition | actual=100 | expected=100 | PASS
  ... (28 more checks) ...
Time: 16235 | TEST PASS | 32/32
```

הטסטבנץ' שולח 16 בתים. מקלט UART מדומה דוגם את tx באמצע כל ביט ומשווה את 10 הביטים ל-{1, data, 0}, ובנוסף נבדק ש-done מגיע בדיוק 100 מחזורים אחרי start.



איור 14: tb_uart_phy – זום: שידור הבית 0xAA.

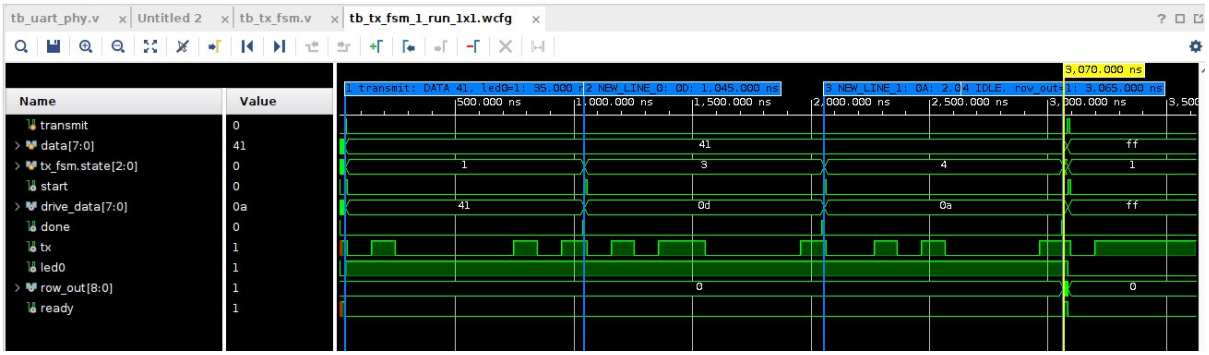
start נטען ל-sh_reg כ-{1, AA, 0} ו-tx יורד ל-(1) start bit. בכל baud_tick, כלומר כל 10 מחזורים, sh_reg זז ימינה ו-cnt עולה; tx מוציא את ביטי הנתון מה-LSB, D0=0 עד (2–9) D7=1, ואחריהם את ה-stop bit (10). ב-baud_tick העשירי done (cnt=9) עולה למחזור אחד והמכונה חוזרת ל-(11) IDLE.

4.2. tb_tx_fsm – בדיקת tx_fsm

פלט הטסטבנץ' (tb_tx_fsm_xsim.log):

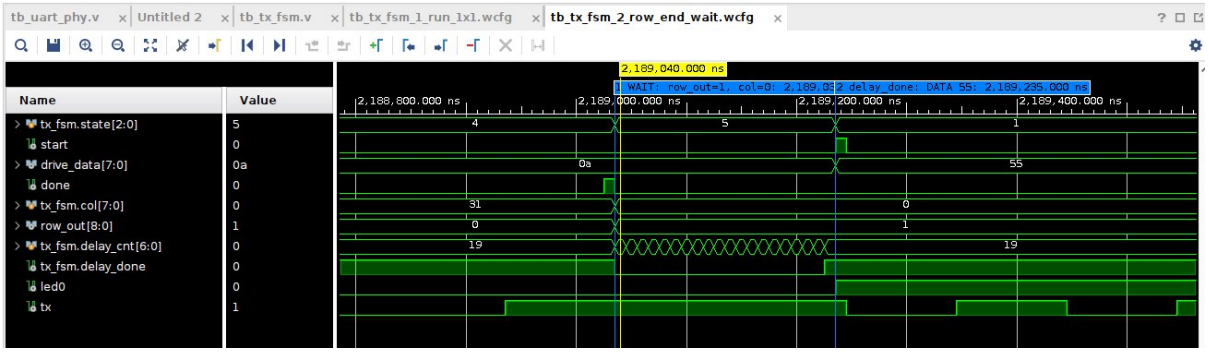
```
Time: 3075 | RUN data=41 delay=0 size=1x1 done | fails so far=0
Time: 6125 | RUN data=ff delay=3 size=1x1 done | fails so far=0
Time: 2117175 | RUN data=20 delay=0 size=32x32 done | fails so far=0
Time: 4422595 | RUN data=55 delay=1 size=32x32 done | fails so far=0
  ... (2 more runs) ...
Time: 9851835 | TEST PASS | pass=8338 | fail=0
```

tx_fsm מחובר ל-uart_phy אמיתי. לכל ריצה הטסטבנץ' בונה את רצף הבתים הצפוי, משווה כל בית ב-start, ובסוף בודק את row_out ואת משך הריצה, כולל זמני ה-WAIT. שורת RUN מודפסת בסוף כל ריצה.



איור 15: tb_tx_fsm – זום: ריצת 1×1 של 0x41.

transmit מעביר את המכונה ל-DATA (מצב 1): start עולה, drive_data=41 ו-led0 נדלק (1). ה-done של הבית מעביר ל-NEW_LINE_0 עם 0D (2) ואחר כך ל-NEW_LINE_1 עם 0A (3). אחרי ה-done האחרון המכונה חוזרת ל-IDLE, ready עולה ו-row_out=1 (4). הסמן הצהוב נמצא בנקודה זו, ועמודת Value מציגה את הערכים בה. בריצת 1×1 אין WAIT, כי הבית היחיד הוא האחרון בשורה האחרונה.



איור 16: tb_tx_fsm – זום: סוף שורה והמצב WAIT, 32×32 עם delay=1.

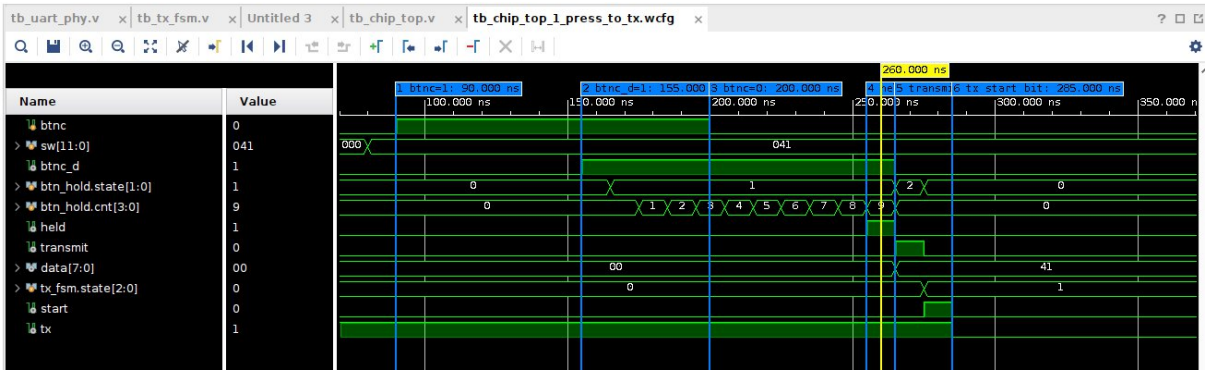
בסוף שידור ה-0A, done מעביר את המכונה מ-NEW_LINE_1 (מצב 4) ל-WAIT (מצב 5): row_out עולה ל-1, col מתאפס ו-delay_cnt מתחיל לספור מ-(1) 0. כש-delay_cnt מגיע ל-19, delay_done עולה, והמכונה עוברת ל-DATA (מצב 1): start עולה, drive_data=55, led0 מתהפך ו-tx יורד ל-start bit של הבית הבא (2). ה-WAIT נמשך 20 מחזורים, שהם DELAY50 בסימולציה.

4.3. tb_chip_top – בדיקת המערכת המלאה

פלט הטסטבנץ' (tb_chip_top_xsim.log):

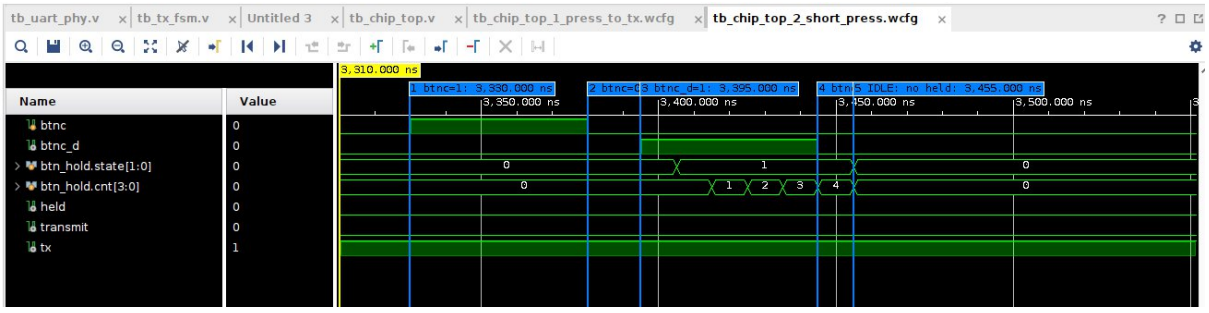
```
Time:   1235 | Check frame (0) | actual=1010000010 | expected=1010000010 | PASS
Time:   2245 | Check frame (1) | actual=1000011010 | expected=1000011010 | PASS
Time:   3255 | Check frame (2) | actual=1000010100 | expected=1000010100 | PASS
Time:   3315 | Transmition ended correctly | PASS
  ... (4175 more checks) ...
Time: 4437105 | TEST PASS | pass=4179 | fail=0
```

הטסטבנץ' מגדיר את המתגים ולוחץ על BTNC. ה-monitor מפענח כל פריים על tx וה-scoreboard משווה אותו לרצף הצפוי. בסוף כל ריצה נבדק שכל הבתים הגיעו ושה-FSM חזר ל-IDLE. בבדיקות השליליות (לחיצה קצרה, לחיצה באמצע ריצה ו-reset) נבדק שלא הגיע אף פריים מיותר.



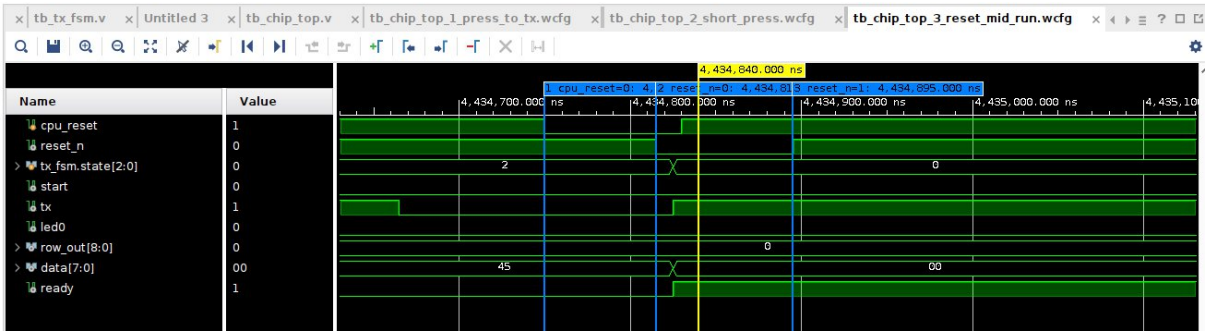
איור 17: tb_chip_top – זום: מהלחיצה ועד ה-start bit.

btnc עולה (1), ו-btnc_d עולה 7 מחזורים אחריו, בסוף הסנכרון וה-(2) debounce. btn_hold.state עובר ל-COUNT (מצב 1) ו-btn_hold.cnt סופר. btnc משתחרר אחרי 11 מחזורים (3), אבל btnc_d נשאר 1 עוד 7 מחזורים, ולכן cnt מגיע ל-9 ו-held עולה (4); הסמן הצהוב נמצא בנקודה זו. במחזור הבא transmit עולה ו-(5) data=41, ו-tx יורד ל-start bit של הבית הראשון (6).



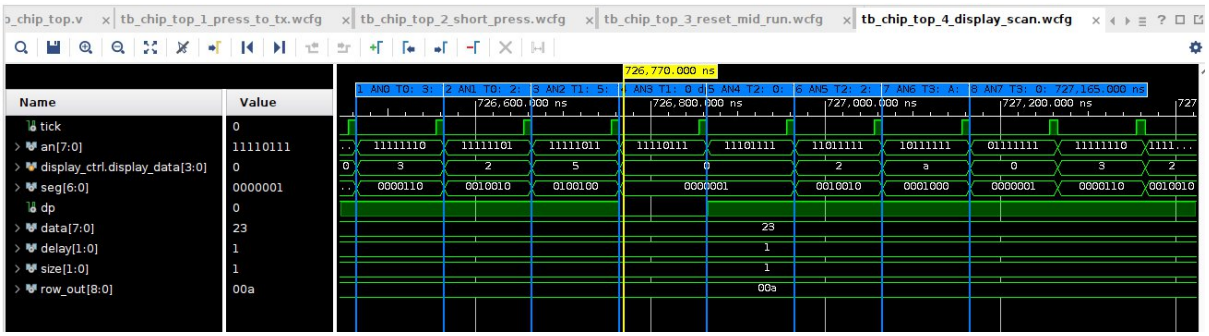
איור 18: tb_chip_top – זום: לחיצה קצרה אינה מפעילה שידור.

לחיצה של 5 מחזורים (1, 2) עוברת את ה-debounce, ו-btnc_d עולה ל-5 מחזורים (3, 4).
btn_hold.state נכנס ל-COUNT, אבל btnc_d יורד כש-cnt=4, הרבה לפני 9, ולכן המכונה חוזרת ל-IDLE בלי (5) held. transmit ו-tx לא משתנים.



איור 19: tb_chip_top – זום: reset באמצע ריצה.

cpu_reset יורד באמצע פריים (1). אחרי ה-debounce reset_n יורד tx_fsm.state :(2) חוזר ל-IDLE ו-data מתאפס; tx ו-ready עולים ל-1, ו-led0 ו-row_out נשארים 0 (הערכים בעמודת Value הם במיקום הסמן הצהוב). reset_n עולה שוב (3) ולא נשלח פריים נוסף. ה-monitor זורק את הפריים שנחתך באמצע.



איור 20: tb_chip_top – זום: סריקת התצוגה, SW=0x23, delay=1, size=1.

בכל tick הטבעת an זזה, ו-display_data בוחר את הספרה של המקום הפעיל (8–1). T0 מציג 23, T1 מציג 0.5, T2 מציג 20 ו-T3 מציג 0A, מספר השורות שנשלחו עד אותו רגע. הסמן הצהוב נמצא על AN3, הספרה היחידה שבה dp=0 (נקודה דולקת).

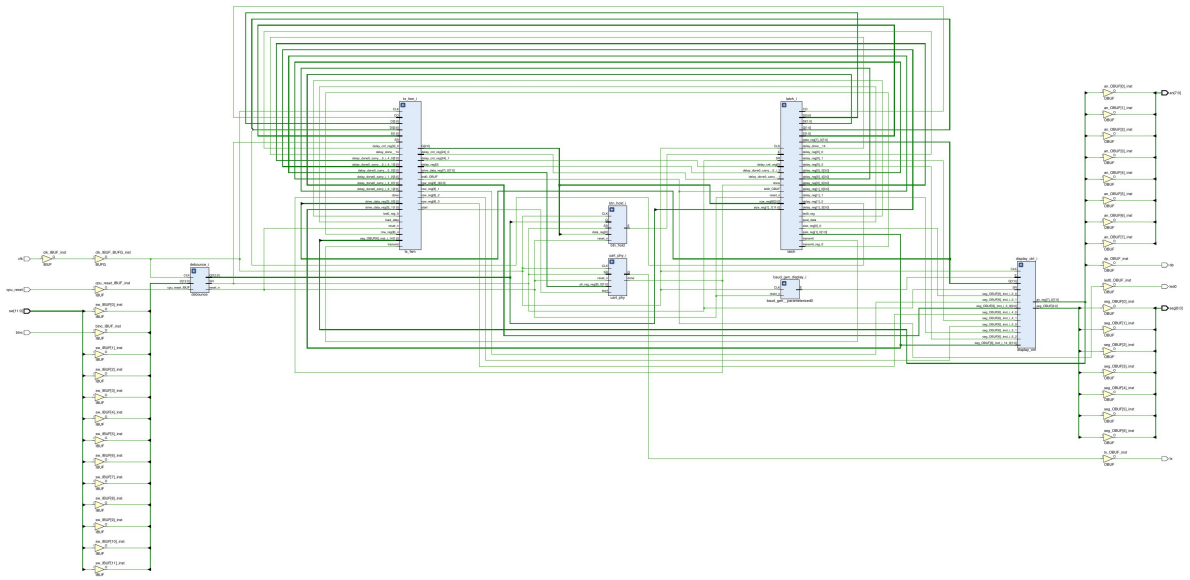
5. סינתזה ומימוש (Synthesis & Implementation)

5.1. סינתזה

הסינתזה (synth_chip_top, Vivado Synthesis Defaults) הסתיימה עם 0 שגיאות, 0 critical warnings ו-0 אזהרות. Vivado זיהה את מכונות המצבים של btn_hold ו-tx_fsm וקידד את tx_fsm מחדש ב-one-hot. בזכות אילוצי ה-false path דוח ה-Methodology נקי.

טבלה 18: תוצאות הסינתזה ודוחות ה-Synthesized Design.

Report	Result
Messages (synthesis.log)	0 errors, 0 critical warnings, 0 warnings
Timing summary (timing_summary_synth.rpt)	sys_clk_pin 10.000 ns (100 MHz): WNS +2.096 ns, TNS 0.000, WHS +0.140 ns, THS 0.000, WPWS +4.500 ns, 0 failing of 636 endpoints – met
DRC (drc_synth.rpt)	0 checks
Methodology (methodology_synth.rpt)	0 checks
Clock networks / interaction	one clock: sys_clk_pin 100 MHz on clk (1 BUFG); only sys_clk_pin → sys_clk_pin paths, classification Clean
Noise (noise_synth.rpt)	18 output pins PASS, 0 FAIL
Power (power_synth.rpt)	0.101 W total on-chip (estimate)



איור 21: סכמת ה-post-synthesis של chip_top.

5.2. מימוש (Implementation)

ה-Implementation (opt_design, place_design, route_design) הסתיים עם 0 שגיאות, 0 critical warnings ו-0 אזהרות. כל הנתיבים עומדים באילוץ של 100MHz עם מרווח של 2.264ns. הנתיב הקריטי עובר מרגיסטר delay ב-latch, דרך ההשוואה של delay_cnt (שרשרת CARRY4), אל ה-enable של drive_data ב-tx_fsm.

טבלה 19: תוצאות המימוש ודוחות ה-Implemented Design.

Report	Result
Messages (implementation.log)	0 errors, 0 critical warnings, 0 warnings
Timing summary (timing_summary_routed.rpt)	WNS +2.264 ns, TNS 0.000 (0 of 663 endpoints failing); WHS +0.122 ns, THS 0.000; WPWS +4.500 ns – all constraints met
Critical path	latch_i/delay_reg[1]/C → tx_fsm_i/drive_data_reg[0]/CE: 7.376 ns, 9 logic levels (CARRY4=6 LUT2=1 LUT4=1 LUT6=1); logic 3.798 ns, route 3.578 ns
Route status (route_status.rpt)	682 logical nets, 432 routable, 432 fully routed, 0 with routing errors
DRC (drc_routed.rpt)	0 checks
Methodology (methodology_drc_routed.rpt)	0 checks
Power (power_routed.rpt)	0.100 W total: 0.097 W static + 0.003 W dynamic; junction 25.5 °C
I/O (io_placed.rpt)	all 33 ports placed on the XDC pins, LVCMOS33 (sw[8], sw[9] LVCMOS18)

סכמות ה-post-implementation זהות לסכמות שאחרי הסינתזה, ולכן אינן מוצגות.

6. ניצול משאבים

הערכים נלקחו מקובצי ה-rpt: utilization_synth.rpt, utilization_placed.rpt. ודוחות ה-out-of-context של כל מודול (תיקייה 5_Resource_Usage).

טבלה 20: ניצול העיצוב המלא לאחר סינתזה ולאחר Place & Route.

Site type	Synthesis	Placed	Available	Util % (placed)
Slice LUTs (all as logic)	170	164	63,400	0.26
Slice Registers (FDRE + FDSE)	246	255	126,800	0.20
Register as Latch	0	0	126,800	0.00
Slices (placed: 62 SLICEL + 37 SLICEM)	–	99	15,850	0.62
F7 / F8 Muxes	0	0	31,700	0.00
Block RAM Tile	0	0	135	0.00
DSPs	0	0	240	0.00
Bonded IOB (15 inputs + 18 outputs)	33	33	210	15.71
BUFGCTRL	1	1	32	3.13
Unique control sets	17	17	15,850	0.11

Primitives – synthesis: FDRE 238, LUT6 55, LUT2 48, CARRY4 41, LUT4 36, LUT5 36, LUT3 19, OBUF 18, LUT1 16, IBUF 15, FDSE 8, BUFG 1 (210 LUT primitives = 170 slice LUTs); placed: FDRE 239, LUT6 55, LUT2 48, CARRY4 41, LUT4 36, LUT5 36, LUT3 19, OBUF 18, FDSE 16, IBUF 15, LUT1 14, BUFG 1 (208 LUT primitives = 164 slice LUTs).

טבלה 21: ניצול משאבים לפי מודול (out-of-context synthesis).

Module	Registers in the RTL	FF	Slice LUTs	Other
debounce	2 x (sync_0, sync, prev, cnt[20:0], output reg); {SW, BTNC} branch 13 bits wide	98	25	12 CARRY4
btn_hold	state[1:0], cnt[26:0]	29	15	7 CARRY4
latch	data[7:0], delay[1:0], size[1:0], transmit	13	3	–
tx_fsm	state, delay_cnt[24:0], col[7:0], row[8:0], drive_data[7:0], start, led0	58	71	18 CARRY4
uart_phy	state, cnt[3:0], sh_reg[9:0] + baud_gen tick_cnt[10:0]	25	27	–
baud_gen	tick_cnt (display: [14:0])	11	13	–
display_ctrl	an[7:0] (incl. seven_segment)	8	35	–
seven_segment	–	0	7	–
Sum of modules	baud_gen counted once (display)	242	189	seven_segment inside display_ctrl
Full design: synth / placed		246 / 255	170 / 164	1 BUFG

לאחר ה-Implementation מספר הדלגלגים עולה מ-246 ל-255, כי opt_design יצר 9 עותקים (lopt_replica) של רגיסטרי an, אחד לכל פין יציאה ועוד אחד ל-dp; סכום המודולים (189 LUTs) גבוה מ-170 LUTs של העיצוב המלא, כי בסינתזה המלאה Vivado מבצע constant propagation ושיתוף לוגיקה מעבר לגבולות המודולים. הניצול מתאים לעיצוב: רוב הדלגלגים הם מונים ארוכים (debounce, החזקה של שנייה והשהיה של עד 200ms), אין BRAM, DSP או latches, והעיצוב תופס פחות מ-1% מה-LUTs ומהדלגלגים של הרכיב.

7. בדיקה על הלוח

7.1. יצירת ה-bitstream וטעינתו ללוח

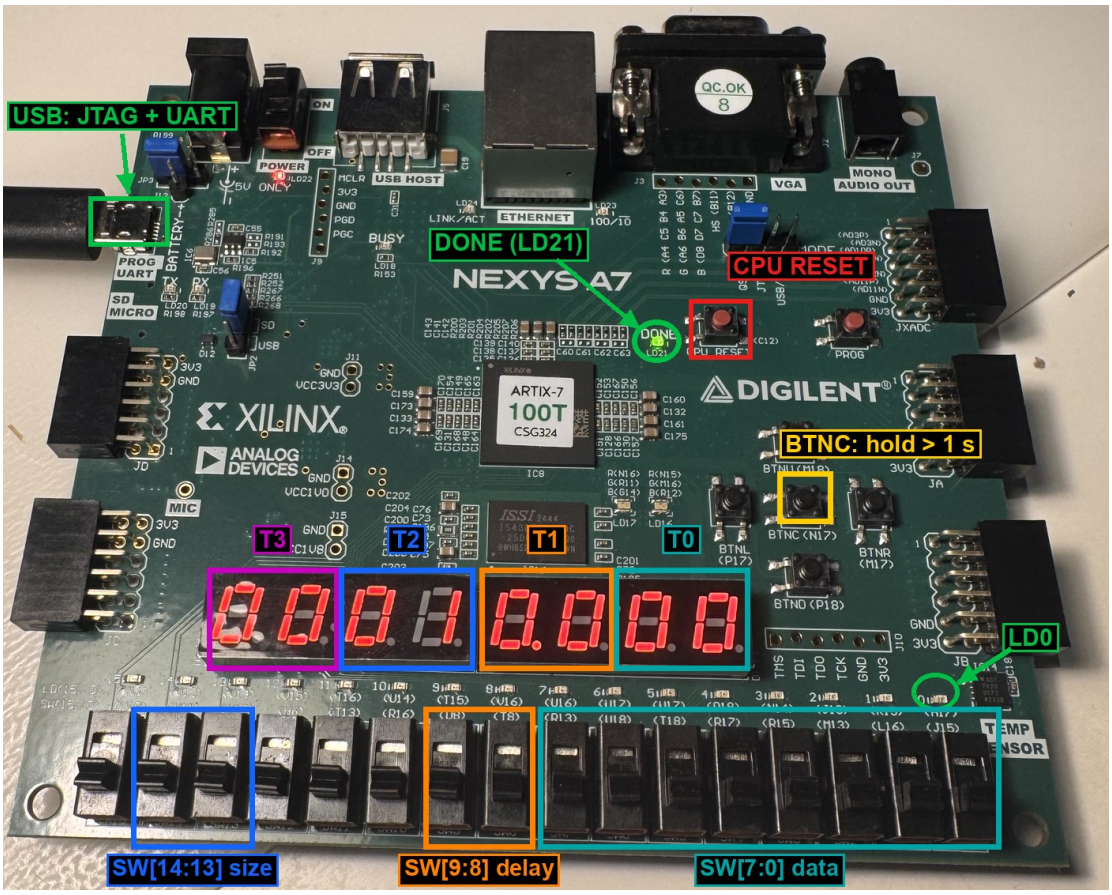
יצירת ה-bitstream (chip_top.bit) הסתיימה עם 0 שגיאות, 0 critical warnings ו-0 אזהרות. הקובץ נטען ללוח דרך JTAG, ונורית ה-DONE נדלקה בסיום הטעינה (איור 22).

טבלה 22: סיכום יצירת ה-bitstream.

Item	Result
Bitstream	chip_top.bit – 0 errors, 0 critical warnings, 0 warnings (6_Bitstream/bitstream_summary.txt)
Device	xc7a100tcsg324-1 (Digilent Nexys A7-100T), Vivado 2025.2
Clock constraint	create_clock sys_clk_pin, period 10.000 ns on clk (E3)
Programming	JTAG over the on-board USB port; DONE LED on
Terminal	tio on macOS, 57600 bps, 8N1, no flow control (USB-UART of the board), log to output.txt

7.2. בדיקה פונקציונלית על הלוח

איור 22 מציג את הלוח מיד אחרי הטעינה. אחרי ה-reset הרגיסטרים של latch מאופסים, ולכן T2=01 (size=0 מפוענח ל-N=1) ושאר השדות 0. כל קבוצת מתגים מסומנת בצבע של השדה שהיא קובעת בתצוגה.



איור 22: הלוח לאחר טעינת ה-bitstream: נורית DONE דולקת והתצוגה במצב ההתחלתי.

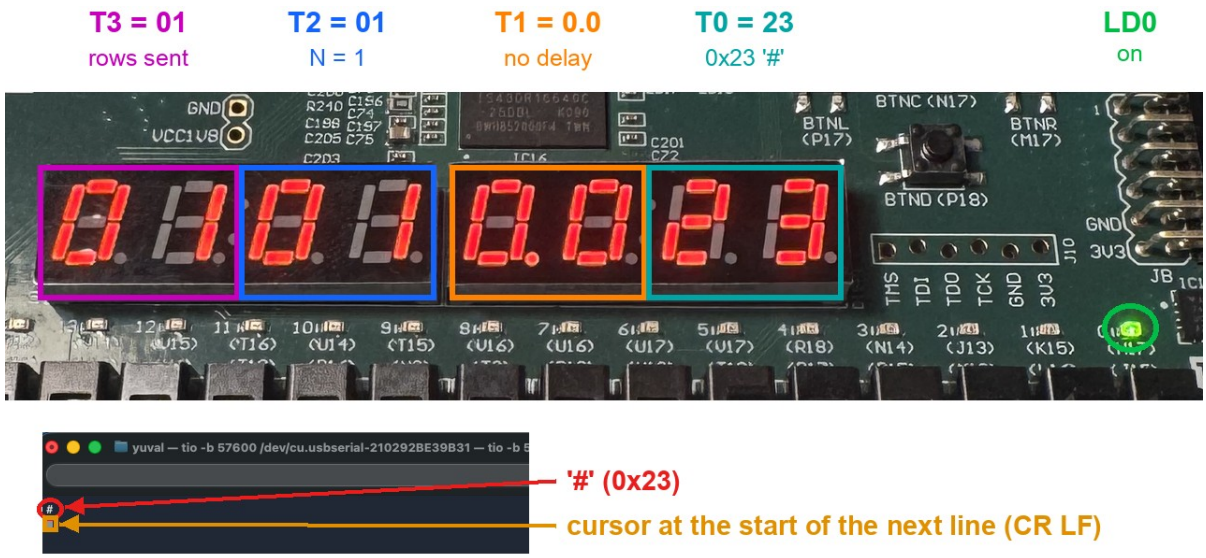
כבל ה-USB של הלוח מספק גם את ממשק ה-JTAG וגם את ערוץ ה-UART. ב-macOS הטרמינל הופעל עם tio, שרושם גם כל תו שמתקבל לקובץ output.txt (תיקייה 7_Board_Test):

```
tio -b 57600 --log --log-file "$PWD/output.txt" --log-strip /dev/cu.usbserial-210292BE39B31
```

לפני כל מקרה הוגדרו המתגים, ואחר כך BTNC הוחזק יותר משנייה. הבית הנשלח הוא תמיד 0x23, התו '#'. בעמודת SW מופיעים SW[7:0] / SW[9:8] / SW[14:13], ובעמודת Display השדות T3 T2 T1 T0 משמאל לימין. טבלה 23 מרכזת את המקרים, ואיורים 23–26 מציגים אותם.

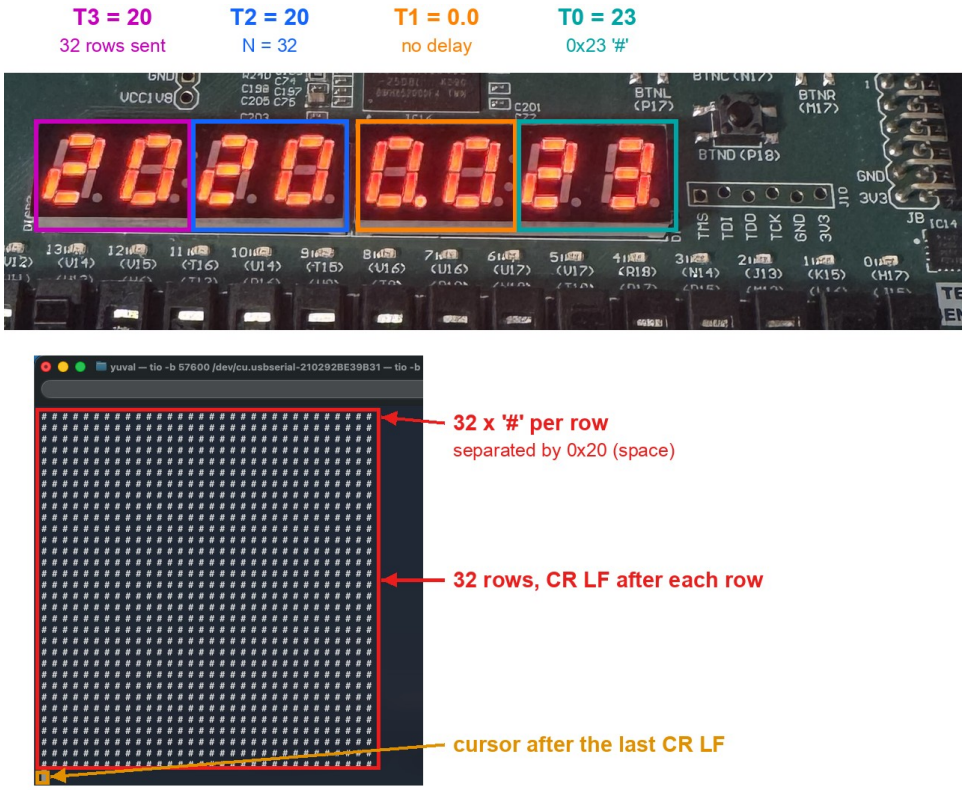
טבלה 23: מקרי הבדיקה על הלוח.

Case	SW	Button	Mode	Display	Result	Figure
1	00 / 00 / 0x23	BTNC > 1 s	1×1, 0 ms	01 01 0.0 23	Pass	23
2	01 / 00 / 0x23	BTNC > 1 s	32×32, 0 ms	20 20 0.0 23	Pass	24
3	11 / 00 / 0x23	BTNC > 1 s	256×256, 0 ms	29 FF 0.0 23 → FF FF 0.0 23	Pass	25
4	01 / 11 / 0x23	BTNC > 1 s	32×32, 200 ms	00 20 2.0 23 → 01 20 2.0 23	Pass	26



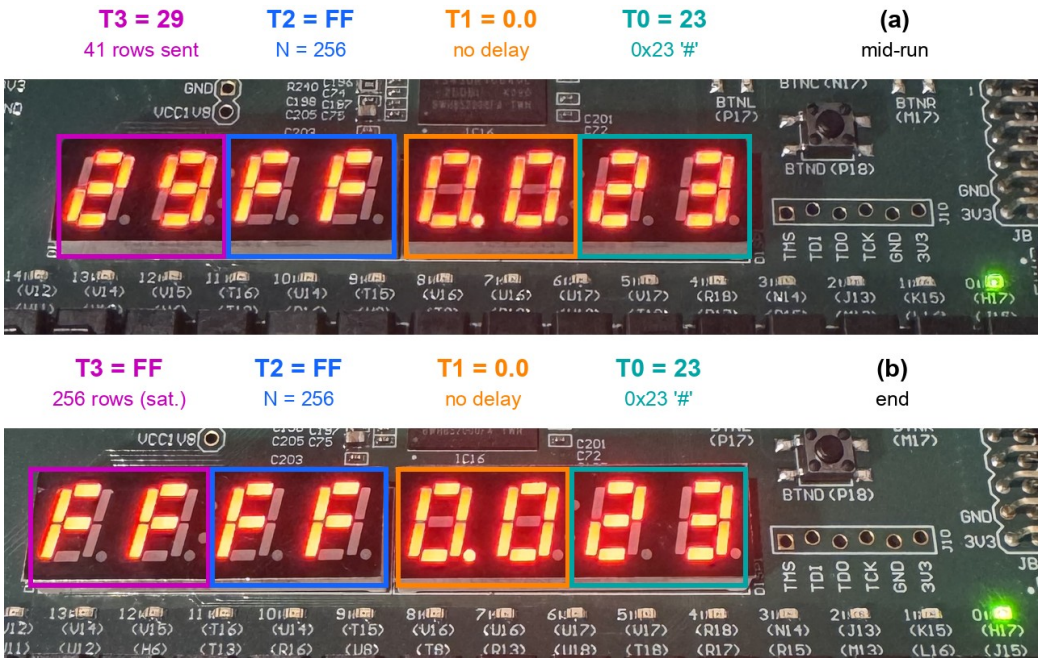
איור 23: מקרה 1, ריבוע 1×1: התצוגה ו-LD0 (למעלה) והטרמינל (למטה).

נשלח '#' יחיד ואחריו CR LF, ולכן הסמן בטרמינל עבר לתחילת השורה הבאה. T3=01 כי נשלחה שורה אחת, ו-LD0 נדלק כי led0 מתהפך בכל בית נתון.



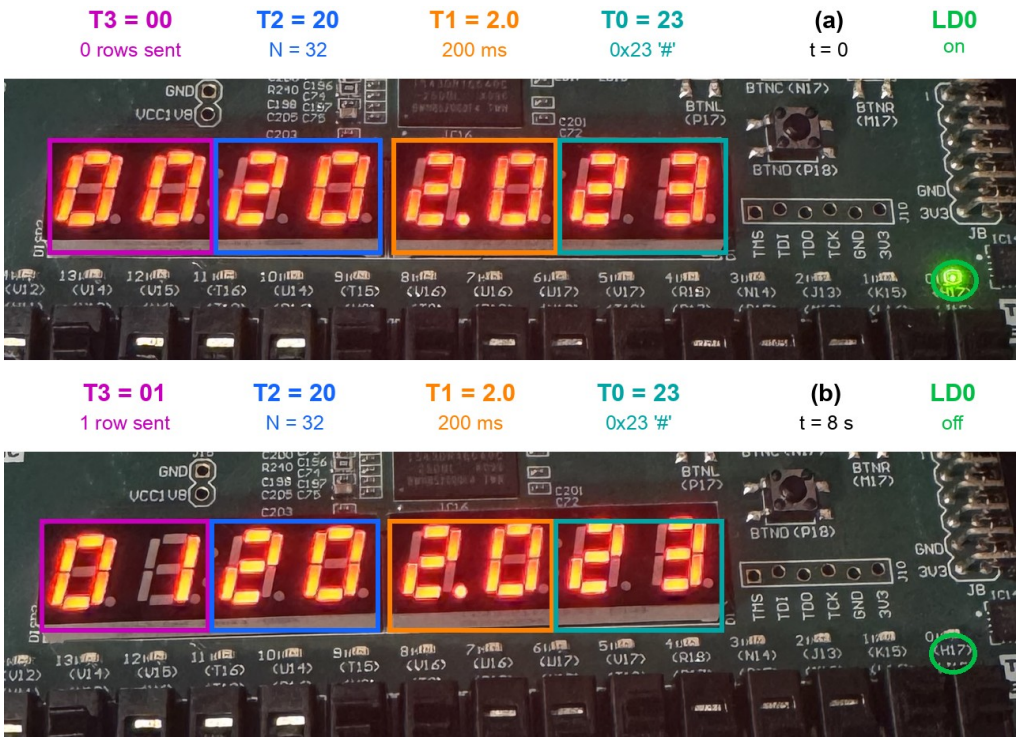
איור 24: מקרה 2, ריבוע 32×32: התצוגה (למעלה) והטרמינל (למטה).

בטרמינל התקבלו 32 שורות של 32 תווי '#' עם רווח ביניהם. בסוף הריצה T3=20, כלומר 32 שורות, כמו N ב-T2.



איור 25: מקרה 3, ריבוע 256×256: (a) באמצע הריצה, (b) בסופה.

באמצע הריצה T3=29, כלומר נשלחו 41 שורות. בסוף הריצה row מגיע ל-256, שלא נכנס בשתי ספרות הקסדצימליות, ולכן התצוגה נעצרת על FF; גם T2 מציג FF עבור N=256. ריבוע של 256 שורות לא נכנס בצילום מסך אחד, ולכן התוכן שלו נבדק בקובץ הלוג (טבלה 24).



איור 26: מקרה 4, ריבוע 32×32 עם השהיה של 200ms; תמונה (b) צולמה 8 שניות אחרי (a).

T1 מציג 2.0, כלומר השהיה של 200ms אחרי כל בית נתון, ולכן שורה של 32 בתים נמשכת כ-6.4 שניות (משוואה 10). בתמונה (b), שצולמה 8 שניות אחרי תמונה (a), המונה T3 עלה רק ל-01. בקצב הזה ההתהפכות של LD0 בכל בית נתון נראית בעין: LD0 דולק ב-(a) וכבוי ב-(b).

$$T_{row} = 32 \cdot 200\ ms + 65 \cdot T_{frame} = 6.4\ s + 11.3\ ms \approx 6.41\ s \qquad (10)$$

קובץ הלוג נרשם בהרצה נוספת אחרי הצילומים, ויש בו 6 ריצות: ריבועים של '#' בכל ארבעת הגדלים וריבועים של '$' (0x24) בגדלים 32 ו-128. הסקריפט check_log.py מחלק את הקובץ לריצות ובודק שבכל ריצה יש N שורות, ושכל שורה מכילה בדיוק N עותקים של הבית עם רווח יחיד ביניהם וללא רווח בסוף השורה. בקובץ אין תווי CR, כי האפשרות log-strip של tio מסננת תווי בקרה; בטרמינל עצמו כל שורה מתחילה בעמודה הראשונה, כלומר ה-CR התקבל.

טבלה 24: בדיקת קובץ הלוג (check_log.py) output.txt.

Run	Byte	Rows × bytes per row	Data bytes	Result
1	0x23 '#'	1 × 1	1	Pass
2	0x23 '#'	32 × 32	1,024	Pass
3	0x23 '#'	128 × 128	16,384	Pass
4	0x23 '#'	256 × 256	65,536	Pass
5	0x24 '$'	32 × 32	1,024	Pass
6	0x24 '$'	128 × 128	16,384	Pass
Total			100,353	Pass

8. סיכום

בקר ה-UART TX מומש בשמונה מודולים סינכרוניים על שעון יחיד של 100MHz: debounce לכל הכניסות, החזקה של שנייה לדגימת המתגים, tx_fsm עם מצב WAIT ויציאות רשומות, uart_phy בקצב 57600bps ותצוגה של 8 ספרות ברענון של 500Hz. שלושת הטסטבנצ'ים עברו (12549 בדיקות, 0 כישלונות), ובצורות הגל נראים הפריים, רצף התווים, ההשהיה, השרשרת מהלחיצה ועד tx, הלחיצה הקצרה, ה-reset וסריקת התצוגה. ה-Lint, הסינתזה, המימוש וה-bitstream נקיים, התזמון נסגר עם מרווח של 2.264ns, והניצול (255 ,164 LUTs דלגלגים) זניח ביחס לרכיב. על הלוח נבדקו ריבועים של 32×32 ,1×1 ו-256×256 וריבוע של 32×32 עם השהיה של 200ms: הטרמינל קיבל את התווים הצפויים, T3 ספר את השורות ונעצר על FF ב-256, ובקובץ הלוג נמצאו 6 ריבועים תקינים (100,353 בתי נתון).

נספח א': קוד המקור (RTL ואילוצים)

chip_top.v

```
`timescale 1ns/1ps
module chip_top #(
    parameter BAUD_RATE = 57600,
    parameter DISPLAY_RR = 4000,
    parameter MAX_COUNT_DEBOUNCE = 2000000,
    parameter MAX_COUNT_BTN_HOLD = 100000000,
    parameter DELAY50 = 5000000
    ) (
    input clk, cpu_reset,
    input btnc,
    input [11:0] sw,
    output tx,
    output led0,
    output [7:0] an,
    output [6:0] seg,
    output dp
);

    wire [11:0] sw_d;
    wire btnc_d;
    wire reset_n;
    debounce #(.MAX_COUNT(MAX_COUNT_DEBOUNCE)) debounce_i (
        .clk(clk),
        .cpu_reset(cpu_reset),
        .btnc(btnc),
        .sw(sw),
        .sw_d(sw_d),
        .btnc_d(btnc_d),
        .cpu_reset_d(reset_n)
    );

    wire held;
    btn_hold #(.MAX_COUNT(MAX_COUNT_BTN_HOLD)) btn_hold_i (
        .clk(clk),
        .reset_n(reset_n),
        .btn(btnc_d),
        .held(held)
    );

    wire ready;
    wire [7:0] data;
    wire [1:0] delay, size;
    wire transmit;
    latch latch_i (
        .clk(clk),
        .reset_n(reset_n),
        .ready(ready),
        .sw_d(sw_d),
        .held(held),
        .data(data),
        .delay(delay),
        .size(size),
```

```
    .transmit(transmit)
  );

  wire done;
  wire [7:0] drive_data;
  wire start;
  wire [8:0] row_out;
  tx_fsm #(.DELAY50(DELAY50)) tx_fsm_i (
    .clk(clk),
    .reset_n(reset_n),
    .data(data),
    .delay(delay),
    .size(size),
    .transmit(transmit),
    .done(done),
    .drive_data(drive_data),
    .start(start),
    .led0(led0),
    .row_out(row_out),
    .ready(ready)
  );

  uart_phy #(.BAUD_RATE(BAUD_RATE)) uart_phy_i (
    .clk(clk),
    .reset_n(reset_n),
    .data(drive_data),
    .start(start),
    .tx(tx),
    .done(done)
  );

  wire tick;
  baud_gen #(.BAUD_RATE(DISPLAY_RR)) baud_gen_display_i (
    .clk(clk),
    .reset_n(reset_n),
    .en(1'b1),
    .baud_tick(tick)
  );

  display_ctrl display_ctrl_i (
    .clk(clk),
    .reset_n(reset_n),
    .tick(tick),
    .data(data),
    .delay(delay),
    .size(size),
    .row(row_out),
    .an(an),
    .seg(seg),
    .dp(dp)
  );

endmodule
```

debounce.v

```
`timescale 1ns/1ps
module debounce #(parameter MAX_COUNT = 2000000) (
    input clk,
    input cpu_reset,
    input btnc,
    input [11:0] sw,
    output [11:0] sw_d,
    output btnc_d,
    output cpu_reset_d
);

    localparam MAX_COUNT_BITS = $clog2(MAX_COUNT);
    localparam [MAX_COUNT_BITS-1:0] ONE = 1;
    // Debonce, need to wait for 20 ms -> 100M * 20m = 2M = MAX_COUNT
    // Going to use 2 counters, 1 for sw + btnc and 1 from reset (because reset is important buton)
    // 1. Need synchronizer for all the async inputs.
    // 2. Need to detect a change (dual edge detector)
    // 3. once change is detected, start counting, if stable for count == MAX_COUNT - 1 can latch the values to
outputs

    // I'll start with reset debounce so that i can use the reset signal as reset for the other debouncer
    // Synchronizer - reset (Initial valuse for reset regs so we start at reset automatically)
    reg reset_sync = 1'b0;
    reg reset_sync_0 = 1'b0;
    always @(posedge clk) begin
        {reset_sync, reset_sync_0} <= {reset_sync_0, cpu_reset};
    end

    // Edge detector - reset
    reg reset_prev = 1'b0;
    always @(posedge clk) begin
        reset_prev <= reset_sync;
    end
    wire reset_edge = reset_prev ^ reset_sync;

    // Counter - reset
    reg [MAX_COUNT_BITS-1:0] reset_cnt = {MAX_COUNT_BITS{1'b0}};
    wire reset_stable = (reset_cnt == MAX_COUNT - 1);
    always @(posedge clk) begin
        if (reset_edge) reset_cnt <= {MAX_COUNT_BITS{1'b0}};
        else if (!reset_stable) reset_cnt <= reset_cnt + ONE;
    end

    // Output reg - reset
    reg reset_n = 1'b0;
    always @(posedge clk) begin
        if (reset_stable & !reset_edge) reset_n <= reset_sync;
    end

    assign cpu_reset_d = reset_n;

    // Now SW + BTNC (Can use reset now)
    // 1. Synchronizer
    reg [12:0] sw_btn_sync, sw_btn_sync_0;
```

```
    always @(posedge clk) begin
        if (!cpu_reset_d) {sw_btn_sync, sw_btn_sync_0} <= {26{1'b0}};
        else {sw_btn_sync, sw_btn_sync_0} <= {sw_btn_sync_0, {sw, btnc}};
    end

    // 2. Edge detector
    reg [12:0] sw_btn_prev;
    always @(posedge clk) begin
        if (!cpu_reset_d) sw_btn_prev <= {13{1'b0}};
        else sw_btn_prev <= sw_btn_sync;
    end
    wire sw_btn_edge = |(sw_btn_prev ^ sw_btn_sync);

    // 3. Counter
    reg [MAX_COUNT_BITS-1:0] sw_btn_cnt;
    wire sw_btn_stable = sw_btn_cnt == MAX_COUNT - 1;
    always @(posedge clk) begin
        if (!cpu_reset_d | sw_btn_edge) sw_btn_cnt <= {MAX_COUNT_BITS{1'b0}};
        else if (!sw_btn_stable) sw_btn_cnt <= sw_btn_cnt + ONE;
    end

    // Output reg
    reg [12:0] sw_btn_d;
    always @(posedge clk) begin
        if (!cpu_reset_d) sw_btn_d <= {13{1'b0}};
        else if (sw_btn_stable & !sw_btn_edge) sw_btn_d <= sw_btn_sync;
    end

    assign sw_d = sw_btn_d[12:1];
    assign btnc_d = sw_btn_d[0];

endmodule
```

btn_hold.v

```
`timescale 1ns/1ps
// MAX_COUNT = 100M * 1 = 100M

module btn_hold #(parameter MAX_COUNT = 100000000) (
    input clk, reset_n,
    input btn,
    output held
);

    localparam MAX_COUNT_BITS = $clog2(MAX_COUNT);
    localparam [MAX_COUNT_BITS-1:0] ONE = 1;

    localparam IDLE = 0, COUNT = 1, WAIT = 2;
    reg [1:0] state, next;

    reg [MAX_COUNT_BITS-1:0] cnt;

    always @(posedge clk) begin
        if (!reset_n) state <= IDLE;
        else state <= next;
```

```
    end

    always @(*) begin
        next = state;
        case (state)
            IDLE: next = btn ? COUNT : IDLE;
            COUNT: begin
                if (!btn) next = IDLE;
                else next = (cnt == MAX_COUNT - 1) ? WAIT : COUNT;
            end
            WAIT: next = btn ? WAIT : IDLE;
            default: next = IDLE;
        endcase
    end


    always @(posedge clk) begin
        if (!reset_n | state != next) cnt <= {MAX_COUNT_BITS{1'b0}};
        else if (state == COUNT) cnt <= cnt + ONE;
    end

    assign held = (state == COUNT) & btn & (cnt == MAX_COUNT - 1);
endmodule
```

latch.v

```
`timescale 1ns/1ps
module latch (
    input clk, reset_n,
    input ready,
    input [11:0] sw_d,
    input held,
    output reg [7:0] data,
    output reg [1:0] delay,
    output reg [1:0] size,
    output reg transmit
);

    wire load_cfg = held & ready;

    // sw reg
    always @(posedge clk) begin
        if (!reset_n) {size, delay, data} <= 12'b0;
        else if (load_cfg) {size, delay, data} <= {sw_d[11:10], sw_d[9:8], sw_d[7:0]};
    end


    // transmit reg
    always @(posedge clk) begin
        if (!reset_n) transmit <= 1'b0;
        else transmit <= load_cfg;
    end

endmodule
```

tx_fsm.v

```
`timescale 1ns/1ps
module tx_fsm #(parameter DELAY50=5000000)(
    input clk, reset_n,
    input [7:0] data,
    input [1:0] delay,
    input [1:0] size,
    input transmit,
    input done,
    output reg [7:0] drive_data,
    output reg start,
    output reg led0,
    output [8:0] row_out,
    output ready
);

    // Delay Calculation:
    // 1. No delay no need to count anything
    // 2. 50ms -> 100M * 50m = 5,000,000
    // 3. 100ms -> 100M * 100m = 10,000,000
    // 4. 200ms -> 100M * 200m = 20,000,000
    localparam DELAY0=0, DELAY100=2*DELAY50, DELAY200=4*DELAY50;
    localparam DELAY_BITS = $clog2(DELAY200+1); // to cover all delay types i took max(DELAYS)
    localparam [DELAY_BITS-1:0] DELAY_ONE = 1;

    localparam SIZE0 = 1, SIZE1 = 32, SIZE2 = 128, SIZE3 = 256;
    localparam SIZE_BITS = $clog2(SIZE3);
    localparam [SIZE_BITS-1:0] SIZE_ONE = 1;

    localparam [7:0] SPACE_DATA = 8'h20, NEW_LINE_0_DATA = 8'h0D, NEW_LINE_1_DATA = 8'h0A;

    localparam IDLE = 0, DATA = 1, SPACE = 2, NEW_LINE_0 = 3, NEW_LINE_1 = 4, WAIT = 5;

    // Registers
    reg [2:0] state, next;
    reg [DELAY_BITS-1:0] delay_reg;
    reg [SIZE_BITS-1:0] size_reg;
    reg [DELAY_BITS-1:0] delay_cnt;
    reg [SIZE_BITS-1:0] col;
    reg [SIZE_BITS:0] row;

    //Decoders
    // 1. delay decodet:
    always @(*) begin
        case (delay)
            2'd0: delay_reg = DELAY0;
            2'd1: delay_reg = DELAY50;
            2'd2: delay_reg = DELAY100;
            2'd3: delay_reg = DELAY200;
        endcase
    end

    // 2. size decoder:
```

```
always @(*) begin
   case(size)
      2'd0: size_reg = SIZE0-1;
      2'd1: size_reg = SIZE1-1;
      2'd2: size_reg = SIZE2-1;
      2'd3: size_reg = SIZE3-1;
   endcase
end

// Signals
wire last_col = (col == size_reg);
wire last_row = (row == size_reg);

// Counters
wire delay_done = (delay_cnt == delay_reg - 1) | (delay_reg == DELAY0);
// signal do reset wait counter:
// 1. when coming from SPACE: state == SPACE & done
// 2. when coming from NEW_LINE1: state == NEW_LINE1 & done & !last_row;
wire rst_delay_count = ((state == SPACE) & done) | ((state == NEW_LINE_1) & done & !last_row);
always @(posedge clk) begin
   if (!reset_n | rst_delay_count) delay_cnt <= {DELAY_BITS{1'b0}};
   else if (!delay_done) delay_cnt <= delay_cnt + DELAY_ONE;
end

wire rst_counters = (state == IDLE) & transmit;
// col counter needs to +1 from transition from SPACE TO WAIT
wire col_transtition = (state == SPACE) & done & !last_col;
// row counter needs to up one from NEW_LINE_1 transition to WAIT
wire row_transtition = (state == NEW_LINE_1) & done;
always @(posedge clk) begin
   if (!reset_n | rst_counters | row_transtition) col <= {SIZE_BITS{1'b0}};
   else if (col_transtition) col <= col + SIZE_ONE;
end

always @(posedge clk) begin
   if (!reset_n | rst_counters) row <= {(SIZE_BITS+1){1'b0}};
   else if (row_transtition) row <= row + {1'b0, SIZE_ONE};
end

//FSM
always @(posedge clk) begin
   if (!reset_n) state <= IDLE;
   else state <= next;
end

always @(*) begin
   next = state;
   case (state)
      IDLE: next = transmit ? DATA : IDLE;
      DATA: if (done) next = (last_col) ? NEW_LINE_0 : SPACE;
      SPACE: next = (done) ? WAIT : SPACE;
      NEW_LINE_0: next = (done) ? NEW_LINE_1 : NEW_LINE_0;
      NEW_LINE_1: if (done) next = (last_row) ? IDLE : WAIT;
      WAIT: next = (delay_done) ? DATA : WAIT;
      default: next = state;
   endcase
```

```
    end

    // Registered outputs
    wire load_data = ((state == IDLE) & transmit) | ((state == WAIT) & delay_done);
    wire load_space= (state == DATA) & !last_col & done;
    wire load_new_line_0 = (state == DATA) & last_col & done;
    wire load_new_line_1 = (state == NEW_LINE_0) & done;
    always @(posedge clk) begin
        if (!reset_n) {start, drive_data, led0} <= {10{1'b0}};
        else begin
            start <= (load_data | load_space | load_new_line_0 | load_new_line_1);
            if (load_data) {drive_data, led0} <= {data, !led0};
            else if (load_space) drive_data <= SPACE_DATA;
            else if (load_new_line_0) drive_data <= NEW_LINE_0_DATA;
            else if (load_new_line_1) drive_data <= NEW_LINE_1_DATA;
        end
    end
    assign row_out = row;
    assign ready = (state == IDLE);
endmodule
```

uart_phy.v

```
`timescale 1ns/1ps
module uart_phy #(parameter BAUD_RATE = 57600) (
    input clk, reset_n,
    input [7:0] data,
    input start,
    output tx,
    output done
);

    // How it works:
    // 2 States, IDLE and TRANSMIT
    // tx is 1 all the time when not in transmit
    // to move to transmit state we need start (then when in transmit -> pass the state to the baud_gen as en so it starts ticking)
    // done is asserted when transmitted bits counter is 9 and there is a tick
    // Counter counter from 0-9 only in transmit and only when there is a tick to make sure that we space the bits in the correct timing
    // Shift register 10 bits, in IDLE we load it with {10{1'b1}} so line is always 1, and when start we load it with {1'b1, data, 1'b0},
    // then for every count enable is acts as a shift en too, and we shift right while inserting a one
    // {1'b1, sh_reg[9:1]}


    // States + state regs
    localparam IDLE = 0, TRANSMIT = 1;
    reg state, next;


    // tick generator
    wire tick;
    baud_gen #(.BAUD_RATE(BAUD_RATE)) baud_gen_i (clk, reset_n, (state == TRANSMIT), tick);
```

```
    // Counter + shift reg
    localparam MAX_COUNT = 10; // count 10 bits
    localparam MAX_COUNT_BITS = $clog2(MAX_COUNT);
    localparam [MAX_COUNT_BITS-1:0] ONE = 1;
    reg [MAX_COUNT_BITS-1:0] cnt;
    reg [MAX_COUNT-1:0] sh_reg;

    //signal we need:
    // 1. To move from IDLE to Transmit we need start, and it is input signal,
    // 2. To move from TRANSMIT to idle: when count is 9 and their is a tick we just transmitted the last bit
(stop bit) so can go back to IDLE
    // 3. shift and count signal, when in state TRANSMIT and tick;
    // 4. signal when to load, when we are in idle and start, and not in transmit
    assign done = (cnt == MAX_COUNT - 1) & tick;
    wire sh_cnt_en = (state == TRANSMIT) & tick;
    wire load = (state == IDLE) & start;

    // counter control
    // 1. needs to be reset to 0.
    // 2. counts only in transmit ans only when their is a tick else sits idle at 0
    always @(posedge clk) begin
        if (!reset_n | done | state == IDLE) cnt <= {MAX_COUNT_BITS{1'b0}};
        else if (sh_cnt_en) cnt <= cnt + ONE;
    end

    // Shift register control
    // 1. reset to ones
    // 2. at start load with data and start stop bit
    // 3. in sh_cnt_ent shift right and insert 1
    always @(posedge clk) begin
        if (!reset_n | done) sh_reg <= {MAX_COUNT{1'b1}};
        else if (load) sh_reg <= {1'b1, data, 1'b0};
        else if (sh_cnt_en) sh_reg <= {1'b1, sh_reg[9:1]};
    end

    //FSM
    always @(posedge clk) begin
        if (!reset_n) state <= IDLE;
        else state <= next;
    end

    always @(*) begin
        next = state;
        case(state)
            IDLE: next = start ? TRANSMIT : IDLE;
            TRANSMIT: next = done ? IDLE : TRANSMIT;
            default: next = state;
        endcase
    end

    // output
    assign tx = sh_reg[0];
endmodule
```

baud_gen.v

```
`timescale 1ns/1ps
module baud_gen #(parameter BAUD_RATE = 57600) (
   input clk, reset_n,
   input en,
   output baud_tick
);

   localparam CLK_FREQ = 100000000;
   localparam COUNT = CLK_FREQ / BAUD_RATE;
   localparam COUNT_BITS = $clog2(COUNT);
   localparam [COUNT_BITS-1:0] ONE = 1;

   reg [COUNT_BITS-1:0] tick_cnt;

   assign baud_tick = (tick_cnt == COUNT - 1);

   always @(posedge clk) begin
     if (!reset_n | baud_tick | !en) tick_cnt <= {COUNT_BITS{1'b0}};
   else tick_cnt <= tick_cnt + ONE;
   end

endmodule
```

display_ctrl.v

```
`timescale 1ns/1ps
module display_ctrl (
   input clk, reset_n,
   input tick,
   input [7:0] data,
   input [1:0] delay,
   input [1:0] size,
   input [8:0] row,
   output reg [7:0] an,
   output [6:0] seg,
   output dp
);

   reg [3:0] display_data;

   // Like in lab 3 need a ring shifter for an
   always @(posedge clk) begin
      if (!reset_n) an <= 8'hFE;
      else if (tick) an <= {an[6:0], an[7]};
   end

   // need to call seven_segment
   seven_segment ss_i (display_data, seg);

   reg [7:0] delay_decode;
   always @(*) begin
      case(delay)
         2'd0: delay_decode = 8'h00;
```

```
            2'd1: delay_decode = 8'h05;
            2'd2: delay_decode = 8'h10;
            2'd3: delay_decode = 8'h20;
        endcase
    end

    reg [7:0] size_decode;
    always @(*) begin
        case(size)
            2'd0: size_decode = 8'h01;
            2'd1: size_decode = 8'h20;
            2'd2: size_decode = 8'h80;
            2'd3: size_decode = 8'hFF;
        endcase
    end

    wire [7:0] row_decode = row[8] ? 8'hFF : row[7:0];

    always @(*) begin
        case(an)
            8'hFE: display_data = data[3:0];
            8'hFD: display_data = data[7:4];
            8'hFB: display_data = delay_decode[3:0];
            8'hF7: display_data = delay_decode[7:4];
            8'hEF: display_data = size_decode[3:0];
            8'hDF: display_data = size_decode[7:4];
            8'hBf: display_data = row_decode[3:0];
            8'h7F: display_data = row_decode[7:4];
            default: display_data = 4'h0;
        endcase
    end


    assign dp = an[3];

endmodule
```

seven_segment.v

```
`timescale 1ns/1ps
module seven_segment (
    input [3:0] dig_in,
    output reg [6:0] ss_dig
);

    always @(*) begin
        case (dig_in)
            4'h0: ss_dig = 7'b0000001;
            4'h1: ss_dig = 7'b1001111;
            4'h2: ss_dig = 7'b0010010;
            4'h3: ss_dig = 7'b0000110;
            4'h4: ss_dig = 7'b1001100;
            4'h5: ss_dig = 7'b0100100;
            4'h6: ss_dig = 7'b0100000;
            4'h7: ss_dig = 7'b0001111;
```

```
        4'h8: ss_dig = 7'b0000000;
        4'h9: ss_dig = 7'b0000100;
        4'hA: ss_dig = 7'b0001000;
        4'hB: ss_dig = 7'b1100000;
        4'hC: ss_dig = 7'b0110001;
        4'hD: ss_dig = 7'b1000010;
        4'hE: ss_dig = 7'b0110000;
        4'hF: ss_dig = 7'b0111000;
        default: ss_dig = 7'b1111111;
      endcase
  end

endmodule
```

NexysA7-100T_Master.xdc (האילוצים הפעילים בלבד)

```
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {clk}];
set_false_path -from [get_ports {cpu_reset btnc sw[*]}]
set_false_path -to [get_ports {tx led0 an[*] seg[*] dp}]
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { sw[0] }]; #IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { sw[1] }]; #IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { sw[2] }]; #IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { sw[3] }]; #IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { sw[4] }]; #IO_L12N_T1_MRCC_14 Sch=sw[4]
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { sw[5] }]; #IO_L7N_T1_D10_14 Sch=sw[5]
set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { sw[6] }]; #IO_L17N_T2_A13_D29_14 Sch=sw[6]
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { sw[7] }]; #IO_L5N_T0_D07_14 Sch=sw[7]
set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { sw[8] }]; #IO_L24N_T3_34 Sch=sw[8]
set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { sw[9] }]; #IO_25_34 Sch=sw[9]
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { sw[10] }]; #IO_L20P_T3_A08_D24_14 Sch=sw[13]
set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { sw[11] }]; #IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
set_property -dict { PACKAGE_PIN H17   IOSTANDARD LVCMOS33 } [get_ports { led0 }]; #IO_L18P_T2_A24_15 Sch=led[0]
set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { seg[6] }]; #IO_L24N_T3_A00_D16_14 Sch=ca
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { seg[5] }]; #IO_25_14 Sch=cb
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { seg[4] }]; #IO_25_15 Sch=cc
set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { seg[3] }]; #IO_L17P_T2_A26_15 Sch=cd
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { seg[2] }]; #IO_L13P_T2_MRCC_14 Sch=ce
```

```
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { seg[1] }];
#IO_L19P_T3_A10_D26_14 Sch=cf
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { seg[0] }]; #IO_L4P_T0_D04_14
Sch=cg
set_property -dict { PACKAGE_PIN H15   IOSTANDARD LVCMOS33 } [get_ports { dp }];
#IO_L19N_T3_A21_VREF_15 Sch=dp
set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { an[0] }];
#IO_L23P_T3_FOE_B_15 Sch=an[0]
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { an[1] }];
#IO_L23N_T3_FWE_B_15 Sch=an[1]
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { an[2] }];
#IO_L24P_T3_A01_D17_14 Sch=an[2]
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { an[3] }]; #IO_L19P_T3_A22_15
Sch=an[3]
set_property -dict { PACKAGE_PIN P14   IOSTANDARD LVCMOS33 } [get_ports { an[4] }]; #IO_L8N_T1_D12_14
Sch=an[4]
set_property -dict { PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { an[5] }];
#IO_L14P_T2_SRCC_14 Sch=an[5]
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { an[6] }]; #IO_L23P_T3_35
Sch=an[6]
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { an[7] }];
#IO_L23N_T3_A02_D18_14 Sch=an[7]
set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { cpu_reset }];
#IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { btnc }]; #IO_L9P_T1_DQS_14
Sch=btnc
set_property -dict { PACKAGE_PIN D4    IOSTANDARD LVCMOS33 } [get_ports { tx }]; #IO_L11N_T1_SRCC_35
Sch=uart_rxd_out
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

נספח ב': קוד הטסטבנצ'ים

הטסטבנצ'ים לפי הסדר של טבלה 15:

tb_uart_phy.v

```
`timescale 1ns/1ps
module tb_uart_phy();

  localparam TEST_BAUD = 10000000;
  localparam COUNT = 100000000 / TEST_BAUD;
  localparam CYCLES_DURING_TRANS = 9*COUNT + COUNT;
  localparam HALF_COUNT = COUNT / 2;

  reg clk = 1'b0;
  always #5 clk = !clk;

  reg reset_n;
  reg [7:0] data;
  reg start;

  wire tx, done;

  reg [7:0] exp_data;
  wire [9:0] exp_frame = {1'b1, exp_data, 1'b0};
  reg [9:0] recived_frame;
  integer pass = 0, fail = 0;

  uart_phy #(.BAUD_RATE(TEST_BAUD)) uart_phy_i (clk, reset_n, data, start, tx, done);

  task reset ();
    begin
      reset_n = 1'b0; data = 8'b0; start = 1'b0; exp_data = 8'b0;
      repeat (2) @(negedge clk);

      reset_n = 1'b1;
    end
  endtask

  task send_byte (input [7:0] driven_data);
    integer cycles_during_trans;
    reg res;
    begin
      cycles_during_trans = 0;

      @(negedge clk); data = driven_data; exp_data = driven_data; start = 1'b1;

      @(negedge clk) start = 1'b0;

      while (!done & cycles_during_trans < 2*CYCLES_DURING_TRANS)  begin
        @(posedge clk) cycles_during_trans = cycles_during_trans + 1;
      end

      res = (cycles_during_trans == CYCLES_DURING_TRANS);
      $display("Time: %0d | Clock cycles during transmition | actual=%0d | expected=%0d | %s", $time,
cycles_during_trans, CYCLES_DURING_TRANS, res ? "PASS" : "FAIL");
```

```
        if (res) pass = pass + 1;
        else fail = fail + 1;
      end
    endtask

    task send_byte_random();
      begin
        send_byte($urandom);
      end
    endtask

    // check we trans in the midle of bit frame
    always @(negedge tx) begin
      uart_rx(recived_frame);
      compare_frames(recived_frame, exp_frame);
    end

    task uart_rx (output [9:0] recived);
      integer i;
      begin
        repeat (HALF_COUNT) @(posedge clk);

        for (i = 0; i < 9; i = i + 1) begin
          recived = {tx, recived[9:1]};
          repeat (COUNT) @(posedge clk);
        end
        recived = {tx, recived[9:1]};
      end
    endtask

    task compare_frames (input [9:0] recived_frame, input [9:0] exp_frame);
      reg res;
      begin
        res = (recived_frame === exp_frame);
        $display("Time: %0d | Frame correctly recived | actual=%0b | expected=%0b | %s", $time,
recived_frame, exp_frame, res ? "PASS" : "FAIL");
        if (res) pass = pass + 1;
        else fail = fail + 1;
      end
    endtask

    initial begin
      $display("Time: %0d | Reset DUT", $time);
      reset();

      $display("Time: %0d | Direct TEST", $time);
      send_byte(8'h00);
      send_byte(8'hAA);
      send_byte(8'h55);
      send_byte(8'hFF);

      $display("Time: %0d | Random TEST", $time);
      repeat (10) send_byte_random();

      $display("Time: %0d | Test transmit after pause", $time);
      send_byte(8'h12);
```

```
    repeat(5) @(negedge clk);
    send_byte(8'h34);

    if (fail  == 0 & pass > 0) $display("Time: %0d | TEST PASS | %0d/%0d", $time, pass, pass + fail);
    else $display("Time: %0d | TEST FAIL | %0d/%0d", $time, pass, pass + fail);

    $finish();
  end

endmodule
```

tb_tx_fsm.v

```
`timescale 1ns/1ps
module tb_tx_fsm ();

  localparam TEST_DELAY = 20;
  localparam TEST_RATE = 10000000;
  localparam FRAME = 10 * (100000000 / TEST_RATE); // cycles of one byte on the line

  reg clk = 1'b0;
  always #5 clk = !clk;

  reg reset_n, transmit;
  reg [7:0] data;
  reg [1:0] delay, size;
  wire done, start, led0, ready, tx;
  wire [7:0] drive_data;
  wire [8:0] row_out;

  integer pass = 0, fail = 0;
  integer cycle_cnt = 0, first_start, last_start;

  tx_fsm #(.DELAY50(TEST_DELAY)) tx_fsm_i (clk, reset_n, data, delay, size, transmit, done, drive_data, start,
led0, row_out, ready);
  uart_phy #(.BAUD_RATE(TEST_RATE)) uart_phy_i (clk, reset_n, drive_data, start, tx, done);

  always @(posedge clk) cycle_cnt = cycle_cnt + 1;

  // wait for the next start pulse from tx_fsm and check the byte it sends to the phy
  task expect_byte (input [7:0] exp);
    begin
      @(posedge clk);
      while (!start) @(posedge clk);
      last_start = cycle_cnt;
      if (drive_data !== exp) begin
        $display("Time: %0d | byte | actual=%h | expected=%h | FAIL", $time, drive_data, exp);
        fail = fail + 1;
      end else pass = pass + 1;
    end
  endtask

  task run (input [7:0] d, input [1:0] dly, input [1:0] sz);
    integer n, dly_cycles, row, col, exp_time;
```

```
    begin
      n = (sz == 0) ? 1 : (sz == 1) ? 32 : (sz == 2) ? 128 : 256;
      dly_cycles = (dly == 0) ? 0 : (dly == 1) ? TEST_DELAY : (dly == 2) ? 2*TEST_DELAY : 4*TEST_DELAY;

      // 1. drive config + 1 cycle transmit pulse
      @(negedge clk); data = d; delay = dly; size = sz; transmit = 1'b1;
      @(negedge clk); transmit = 1'b0;

      // 2. expected stream: data, then space or (0D, 0A) at the end of the row
      for (row = 0; row < n; row = row + 1) begin
        for (col = 0; col < n; col = col + 1) begin
          expect_byte(d);
          if (row == 0 && col == 0) first_start = last_start;
          if (col == n - 1) begin
            expect_byte(8'h0D);
            expect_byte(8'h0A);
          end else expect_byte(8'h20);
        end
      end

      // 3. wait for the FSM to go back to IDLE
      while (!ready) @(posedge clk);

      // 4. row counter
      if (row_out !== n) begin
        $display("Time: %0d | row_out | actual=%0d | expected=%0d | FAIL", $time, row_out, n);
        fail = fail + 1;
      end else pass = pass + 1;

      // 5. run time (delay check): every byte takes FRAME + 1 cycles,
      //    and before every data byte (except the first) the FSM waits max(delay, 1) more cycles
      exp_time = (n * (2*n + 1) - 1) * (FRAME + 1) + (n * n - 1) * ((dly_cycles > 1) ? dly_cycles : 1);
      if (last_start - first_start !== exp_time) begin
        $display("Time: %0d | run time | actual=%0d | expected=%0d | FAIL", $time, last_start - first_start,
exp_time);
        fail = fail + 1;
      end else pass = pass + 1;

      $display("Time: %0d | RUN data=%h delay=%0d size=%0dx%0d done | fails so far=%0d", $time, d,
dly, n, n, fail);
    end
  endtask

  initial begin
    reset_n = 1'b0; transmit = 1'b0; data = 0; delay = 0; size = 0;
    repeat (2) @(negedge clk);
    reset_n = 1'b1;

    run(8'h41, 2'd0, 2'd0); // 1x1
    run(8'hFF, 2'd3, 2'd0); // 1x1, max delay
    run(8'h20, 2'd0, 2'd1); // 32x32, data == space
    run(8'h55, 2'd1, 2'd1); // 32x32, 50ms
    run(8'hAA, 2'd2, 2'd1); // 32x32, 100ms
    run(8'h0D, 2'd3, 2'd1); // 32x32, 200ms
    //run(8'h12, 2'd0, 2'd2); // 128x128
```

```
        $display("Time: %0d | TEST %s | pass=%0d | fail=%0d", $time, (fail == 0) ? "PASS" : "FAIL", pass, fail);
        $finish;
    end

endmodule
```

tb_chip_top.v

```
`timescale 1ns/1ps
module tb_chip_top ();

    localparam MAX_COUNT_BTN_HOLD = 10;
    localparam MAX_COUNT_DEBOUNCE = 4;
    localparam BAUD_RATE = 10000000;
    localparam CYCLES_PER_BIT = 100000000 / BAUD_RATE;
    localparam DISPLAY_RR = 10000000;
    localparam DELAY50 = 20;

    localparam HALF_BIT = CYCLES_PER_BIT / 2;
    localparam SYNC_CYCLES = 2;

    /*
        Timing Calculation for button press:
        1. debounce.v:
            from moment of pressing until btn_d: SYNC_CYCLES(2) + EDGE_CYCLE(1) + (MAX_COUNT_DEBOUNCE-
1) + REG_CYCLE(1) = 2+1+3+1 = 7

        2. btn_hld.v:
            from moment of btn unitl held: STATE_CYCLE(1) + (MAX_COUNT_BTN_HOLD-1) = 1 + 9 = 10

        3. latch.v
            from moment of held until transmit: LATCH_CYCLE = 1
        total latencey is 1 + 10 + 7 = 18 clk cycles.
        In order to asure a press Need minimum of 11 cycles pressed for btn hold:
            L = 18 - (debounce cycle = 7) - (latch-cycles = 1) + (one cycle for held = 1) = 18 - 7 -1 + 1 = 11;
            MIN_HOLD = STATE_CYCLE + (MAX_COUNT_BTN_HOLD-1) + 1(for held itself) = 11

        The min hold time to get a start count in btn hold but not transmit is: EDGE_CYCLE +
(MAX_CDEBOUNCE-1) + 1 = 5
        so SHORT_HOLD = EDGE_CYCLE + (MAX_COUNT_DEBOUNCE-1) + 1 = 5;
        and LONG_HOLD = STATE_CYCLE + (MAX_COUNT_BTN_HOLD-1) + 1 = 11;
    */

    localparam SHORT_HOLD = MAX_COUNT_DEBOUNCE + 1;
    localparam LONG_HOLD = MAX_COUNT_BTN_HOLD + 1;

    localparam FRAME_CYCLES = 10 * CYCLES_PER_BIT;
    localparam TX_LATENCY = (SYNC_CYCLES + 1 + (MAX_COUNT_DEBOUNCE-1) + 1) + (1 +
(MAX_COUNT_BTN_HOLD-1)) + 1 + 2;
    localparam NO_TX_WAIT = TX_LATENCY + 2 * FRAME_CYCLES;

    reg clk = 1'b0;
    always #5 clk = !clk;
```

```
    reg cpu_reset, btnc;
    reg [11:0] sw;

    wire tx, led0, dp;
    wire [7:0] an;
    wire [6:0] seg;

    integer pass = 0, fail = 0;

    chip_top
#(.BAUD_RATE(BAUD_RATE), .DISPLAY_RR(DISPLAY_RR), .MAX_COUNT_DEBOUNCE(MAX_COUNT_DEBOUNCE
), .MAX_COUNT_BTN_HOLD(MAX_COUNT_BTN_HOLD), .DELAY50(DELAY50)) dut (
        .clk(clk),
        .cpu_reset(cpu_reset),
        .btnc(btnc),
        .sw(sw),
        .tx(tx),
        .led0(led0),
        .an(an),
        .seg(seg),
        .dp(dp)
    );

    /*
        so the dut gets size if 0 -> 1, 1->32, 2->128, 3-256
        the max size of exp_mem (includes all the transmitions):
        1. each row has size*data + (size-1)*space + 2(for new line) => 2*size + 1;
        2. their are size cols
        so in total size * (2*size + 1) elments in transmition
    */

    localparam MAX_SIZE = 256;
    integer wr_ptr = 0, rd_ptr = 0;
    reg [7:0] exp_mem [0 : MAX_SIZE * (2*MAX_SIZE + 1) - 1];

    task power_on();
        begin
            cpu_reset = 1'b1; btnc = 1'b0; sw = 12'b0;
            while (dut.reset_n !== 1'b1) @(posedge clk);
        end
    endtask

    task reset();
        begin
            @(negedge clk) cpu_reset = 1'b0;
            while (dut.reset_n !== 1'b0) @(posedge clk);
            @(negedge clk); cpu_reset = 1'b1;
            while (dut.reset_n !== 1'b1) @(posedge clk);
        end
    endtask

    task btn_press(input integer cycles_to_press);
        begin
            @(negedge clk); btnc = 1'b1;
            repeat(cycles_to_press) @(posedge clk);
            @(negedge clk) btnc = 1'b0;
```

```
    end
  endtask

  reg reset_in_frame = 1'b0;
  always @(negedge dut.reset_n) reset_in_frame = 1'b1;

  reg [9:0] tx_frame;
  reg res_frame;
  always @(negedge tx) begin
    if (dut.reset_n) begin
      reset_in_frame = 1'b0;
      build_byte(); // Got the frame
      if (reset_in_frame) begin
        $display("Time: %7d | Frame cut by reset | dropped", $time);
      end else begin
        // need to check if frame is correct copared to exp_mem
        if (rd_ptr >= wr_ptr) begin
          fail = fail + 1;
          $display("Time: %7d | ERROR Unexpected frame | wr_ptr=%0d | rd_ptr=%0d | FAIL", $time,
wr_ptr, rd_ptr);
        end else begin
          res_frame = (tx_frame === {1'b1, exp_mem[rd_ptr], 1'b0});
          $display("Time: %7d | Check frame (%0d) | actual=%b | expected=%b | %s", $time, rd_ptr,
tx_frame, {1'b1, exp_mem[rd_ptr], 1'b0}, res_frame ? "PASS" : "FAIL");
          if (res_frame) pass = pass + 1;
          else fail = fail + 1;
          // incerement to next frame
          rd_ptr = rd_ptr + 1;
        end
      end
    end
  end

  task build_byte();
    integer i;
    begin
      repeat(HALF_BIT) @(posedge clk);
      tx_frame = {tx, tx_frame[9:1]};
      for (i = 0; i < 9; i = i + 1) begin
        repeat(CYCLES_PER_BIT) @(posedge clk);
        tx_frame = {tx, tx_frame[9:1]};
      end
    end
  endtask


  task build_exp(input integer s, input [7:0] d);
    integer row, col;
    begin
      wr_ptr = 0; rd_ptr = 0;
      for (row = 0; row < s; row = row + 1) begin
        for (col = 0; col < s; col = col + 1) begin
          exp_mem[wr_ptr] = d; wr_ptr = wr_ptr + 1;
          if (col == s - 1) begin // if last char in a row print new line
            exp_mem[wr_ptr] = 8'h0D; wr_ptr = wr_ptr + 1;
            exp_mem[wr_ptr] = 8'h0A; wr_ptr = wr_ptr + 1;
```

```
            end else begin
               exp_mem[wr_ptr] = 8'h20; wr_ptr = wr_ptr + 1; // else add a space after the char
            end
         end
      end
   end
endtask

function integer get_size(input [1:0] s);
   case(s)
      2'd0: get_size = 1;
      2'd1: get_size = 32;
      2'd2: get_size = 128;
      2'd3: get_size = 256;
   endcase
endfunction

function integer get_delay(input [1:0] d);
   case(d)
      2'd0: get_delay = 0;
      2'd1: get_delay = DELAY50;
      2'd2: get_delay = 2 * DELAY50;
      2'd3: get_delay = 4 * DELAY50;
   endcase
endfunction

task drive(input [7:0] d, input [1:0] s, input [1:0] delay, input integer press);
   begin
      @(negedge clk); sw = {s, delay, d};
      btn_press(press);
   end
endtask

task wait_done();
   integer transmition_cycles;
   integer cycles;
   integer delay_cycles;
   begin
      delay_cycles = get_delay(sw[9:8]);
      cycles = 0;
      transmition_cycles = wr_ptr * (2 * 10 * CYCLES_PER_BIT + delay_cycles);
      while (!(rd_ptr == wr_ptr & (dut.ready)) & (cycles < transmition_cycles + 1000)) begin
         @(posedge clk); cycles = cycles + 1;
      end

      if (rd_ptr == wr_ptr & dut.ready) begin
         $display("Time: %7d | Transmition ended correctly | PASS", $time);
         pass = pass + 1;
      end else begin
         $display("Time: %7d | Timeout on transmition | FAIL", $time);
         fail = fail + 1;
      end
   end
endtask

task test_transmit(input [7:0] d, input [1:0] s, input [1:0] delay);
```

```
    begin
        build_exp(get_size(s), d);
        drive(d, s, delay, LONG_HOLD);
        wait_done();
    end
endtask

task test_no_transmit(input [7:0] d, input [1:0] s, input [1:0] delay);
    integer fail_before;
    reg res;
    begin
        fail_before = fail;
        drive(d, s, delay, SHORT_HOLD);
        repeat(NO_TX_WAIT) @(posedge clk);
        res = (fail == fail_before) & dut.ready;
        $display("Time: %7d | No transmition on short press | %s", $time, res ? "PASS" : "FAIL");
        if (res) pass = pass + 1;
        else fail = fail + 1;
    end
endtask

task test_press_mid_transmit();
    integer fail_before;
    reg res;
    begin
        build_exp(get_size(2'd1), 8'h46);
        drive(8'h46, 2'd1, 2'd0, LONG_HOLD);
        repeat(1000) @(posedge clk);
        drive(8'h47, 2'd0, 2'd0, LONG_HOLD);
        wait_done();

        fail_before = fail;
        repeat(NO_TX_WAIT) @(posedge clk);
        res = (fail == fail_before) & dut.ready;
        $display("Time: %7d | Press mid transmition ignored | %s", $time, res ? "PASS" : "FAIL");
        if (res) pass = pass + 1;
        else fail = fail + 1;
    end
endtask

task test_reset_mid_transmit();
    integer fail_before;
    reg res;
    begin
        build_exp(get_size(2'd1), 8'h45);
        drive(8'h45, 2'd1, 2'd0, LONG_HOLD);
        repeat(1000) @(posedge clk);
        reset();
        wr_ptr = rd_ptr;
        res = (tx === 1'b1) & (led0 === 1'b0) & (dut.ready === 1'b1) & (dut.row_out === 0);
        $display("Time: %7d | Reset mid transmition | %s", $time, res ? "PASS" : "FAIL");
        if (res) pass = pass + 1;
        else fail = fail + 1;

        fail_before = fail;
        repeat(NO_TX_WAIT) @(posedge clk);
```

```
        res = (fail == fail_before) & (tx === 1'b1);
        $display("Time: %7d | No transmition after reset | %s", $time, res ? "PASS" : "FAIL");
        if (res) pass = pass + 1;
        else fail = fail + 1;
      end
    endtask

    initial begin
      power_on();

      test_transmit(8'h41, 2'd0, 2'd0);
      test_no_transmit(8'h42, 2'd0, 2'd0);
      test_transmit(8'h23, 2'd1, 2'd1);
      test_press_mid_transmit();
      test_reset_mid_transmit();

      $display("Time: %7d | TEST %s | pass=%0d | fail=%0d", $time, (fail == 0) ? "PASS" : "FAIL", pass, fail);
      $finish;
    end
endmodule
```